

UNIVERSIDAD FRANCISCO DE VITORIA

ESCUELA POLITÉCNICA SUPERIOR

COMPUTER SCIENCE DEGREE

FINAL DEGREE PROJECT

RESEARCH TRACK

Word-Level American Sign Language Recognition from Hand Landmarks

Francisco Sandoval Hernández

Tutor: Moisés Martínez Muñoz

CALIFICACIÓN DEL PROYECTO DE FIN DE GRADO

CUALITATIVA:	
NUMÉRICA:	

Conforme Presidente:	**Conforme Secretario:**
Fdo.:	Fdo.:

Conforme Vocal:	**Conforme Vocal:**	**Conforme Vocal:**
Fdo.:	Fdo.:	Fdo.:

Lugar y fecha: Pozuelo de Alarcón, a ____ de _____ de 2026

Acknowledgements

I would like to thank my supervisor, Moisés Martínez Muñoz, for the technical guidance, the steady feedback, and the willingness to push back on every shortcut along the way. The shape of this project owes a great deal to those conversations.

I am also grateful to the Escuela Politécnica Superior of Universidad Francisco de Vitoria for providing the academic environment in which this project took shape, and to the lecturers whose courses laid the foundations on which the rest was built.

Finally, I want to thank my family and friends for their patience and support throughout the long stretches of work that this thesis required.

Abstract

Sign language recognition serves to mitigate communication barriers between deaf or hard-of-hearing signers and non-signing people. This Final Degree Project develops and evaluates a research prototype for word-level American Sign Language (ASL) recognition from short video clips. The work first reviews the accessibility problem, the computer vision methods used for recognition and the technical requirements of the system. It then follows the methodological evolution of the project, from initial approaches based on hand detection and static letter recognition towards a sequence-based model that represents the movement of signs across frames. The final system uses MediaPipe Hands to extract 21 3D hand landmarks per frame, normalises the resulting sequences and classifies each clip with a bidirectional Gated Recurrent Unit (GRU) with a temporal attention mechanism. The experiments evaluate architecture choices, sequence length, cross-dataset evaluation and vocabulary size using the Word-Level American Sign Language (WLASL) and Microsoft American Sign Language (MS-ASL) datasets. The results show consistent performance for reduced vocabularies and across datasets, but accuracy decreases as the number of classes increases. The landmark representation also abstracts away identifying visual information, offering a privacy-conscious alternative to raw video, although the prototype remains a research system rather than a production-ready translator.

Keywords: Sign Language Recognition, Artificial Intelligence, Deep Neural Networks, Computer Vision, Hand Landmarks.

Resumen

El reconocimiento de lengua de signos aborda barreras de comunicación entre personas sordas o con discapacidad auditiva y usuarios que no conocen la lengua de signos. Este Proyecto Fin de Grado desarrolla y evalúa un prototipo de investigación para el reconocimiento de palabras en American Sign Language (ASL) a partir de vídeos breves. El trabajo revisa primero el problema de accesibilidad, los métodos de visión por computador utilizados para el reconocimiento y los requisitos técnicos del sistema. A continuación, recoge la evolución metodológica del proyecto, desde los enfoques iniciales basados en detección de manos y reconocimiento estático de letras hacia un modelo basado en secuencias que representa el movimiento de los signos a lo largo de los fotogramas. El sistema final utiliza MediaPipe Hands para extraer 21 puntos clave 3D de la mano por fotograma, normaliza las secuencias resultantes y clasifica cada vídeo mediante una red recurrente con puertas (GRU) bidireccional con un mecanismo de atención temporal. Los experimentos evalúan las decisiones de arquitectura, la longitud de secuencia, la evaluación entre conjuntos de datos y el tamaño del vocabulario utilizando los conjuntos Word-Level American Sign Language (WLASL) y Microsoft American Sign Language (MS-ASL). Los resultados muestran un comportamiento consistente en vocabularios reducidos y entre conjuntos de datos, pero la precisión disminuye al aumentar el número de clases. La representación mediante puntos clave también descarta la mayor parte de la información visual del vídeo original, lo que limita los datos personales que conserva el sistema, aunque el prototipo sigue siendo un sistema de investigación y no un traductor listo para producción.

Palabras clave: reconocimiento de lengua de signos, Inteligencia Artificial, Redes Neuronales Profundas, Visión por Computador, Puntos de referencia de la mano.

Contents

List of Acronyms **xiii**

1 Introduction **1**
1.1 Problem description . . . 1
1.2 Research scope and approach . . . 1
1.3 Outline . . . 2

2 State of the Art **3**
2.1 From Symbolic to Subsymbolic AI . . . 3
2.2 Deep Neural Networks . . . 4
2.3 Feature Extraction Architectures . . . 5
2.4 Sequence Processing Architectures . . . 8
2.5 Attention Mechanisms . . . 10
2.6 Related Work in Sign Language Recognition . . . 12
2.7 Dataset Limitations . . . 13

3 Objectives **14**
3.1 Main objective . . . 14
3.2 Specific objectives . . . 14
3.3 Validation methods . . . 15

4 Methodology and Work Plan Overview **18**
4.1 Materials and Resources . . . 18
4.2 Methods . . . 20
4.3 Methodology . . . 22
4.4 Work Plan . . . 23
4.5 Constraints and Limitations . . . 26

5 Technical Development **27**
5.1 Solution design . . . 27
5.2 Requirements . . . 32
5.3 Datasets . . . 35

6 Experiments **40**
6.1 Experimental Methodology . . . 40
Part I: Quantitative Evaluation . . . 42

6.2 Phase 1 - Exploratory Work and Methodological Pivot . . . 42
6.3 Phase 2 - Architecture Selection on WLASL30 . . . 44
6.4 Phase 3 - Sequence Length Sweep . . . 47
6.5 Phase 4 - WLASL30 Validation . . . 49
6.6 Phase 5 - Cross-Dataset Validation on MS-ASL50 . . . 50
6.7 Phase 6 - Vocabulary Scaling . . . 52
6.8 Phase 7 - Quantitative Balance and Limitations . . . 54
Part II: Qualitative Evaluation . . . 56
6.9 Qualitative Validation . . . 56

7 Ethical Implications and Social Impact **59**
7.1 Ethical Impact of AI-Based Systems . . . 59
7.2 Ethical Evaluation Framework: Google's AI Principles . . . 59
7.3 Project Analysis Against Google's AI Principles . . . 60
7.4 Impact Scale of Ethical Aspects . . . 63
7.5 Conclusions of the Ethical Analysis . . . 64

8 My Journey at UFV **66**
8.1 The PFG as the Culmination of My University Journey . . . 66
8.2 Connection with My Professional Future . . . 67

9 Conclusions **68**
9.1 General Conclusions . . . 68
9.2 Conclusions regarding Specific Objectives . . . 68
9.3 Next Steps . . . 69

Bibliography **71**

A Additional Experimental Figures **74**
A.1 Large Confusion Matrix . . . 74
A.2 Vocabulary Scaling Diagnostic . . . 76

B User Manual **78**
B.1 Repository Layout . . . 78
B.2 Environment Setup . . . 78
B.3 Dataset Preparation . . . 79
B.4 Training . . . 80
B.5 Evaluation . . . 80
B.6 Inference Server . . . 81
B.7 Mobile Prototype . . . 81
B.8 Smoke Tests . . . 81

C Declaration of the Use of AI Systems **83**

List of Figures

2.1 Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created. . . . 4
2.2 Decomposition of a digital RGB image into its three colour channels. . . . 5
2.3 Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [6]. . . . 6
2.4 AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [4]. . . . 7
2.5 Rendered hand tracking result with hand landmarks and multi-hand tracking output. Source: Zhang et al. [7]. . . . 7
2.6 MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [7]. . . . 8
2.7 Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created. . . . 9
2.8 Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11]. . . . 10
2.9 Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [12]. . . . 11

4.1 Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration and final documentation. Source: self-created with GanttProject. . . . 24

5.1 Solution architecture showing the data preparation, training and evaluation, and run-time inference stages. . . . 28

6.1 Row-normalised confusion matrix of the selected GRU+Attention configuration on the WLASL30 test split (34 samples). Source: self-created from experiment outputs. 46
6.2 Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs. . . . 48

6.3 Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs. 54

A.1 Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). Source: self-created from experiment outputs. 75
A.2 Parallel-coordinates view of the selected GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green for smaller vocabularies to red for larger vocabularies. Source: self-created from experiment outputs. 77

List of Tables

3.1 Validation mapping for the specific objectives. 17

4.1 Main software libraries and tools used in the project. 19

5.1 Operational environment for training, landmark extraction, and runtime inference. . . 31
5.2 FR-001: Landmark extraction from video clips 33
5.3 FR-002: Word-level sign classification 33
5.4 FR-003: Reproducible experiment tracking 33
5.5 FR-004: Multi-seed training support 34
5.6 NFR-001: Inference latency 34
5.7 NFR-002: Modular code architecture 34
5.8 NFR-003: Documentation completeness 35
5.9 Comparative overview of all datasets used in this FDP. 36

6.1 Configuration of the exploratory pilots. Source: self-created from the dataset chapter and stored exploratory artefacts. 43
6.2 Configuration used for architecture selection. Source: self-created from experiment manifests and configuration files. 44
6.3 Single-split test results of the five locally tuned architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the selected configuration is in Section 6.5. 44
6.4 Most relevant confusion examples observed in the selected GRU+Attention WLASL30 run. Source: self-created from stored experiment outputs. 47
6.5 Configuration of the sequence-length sweep. Source: self-created from sweep configuration files. 47
6.6 Sequence-length sweep on WLASL30 with 3 seeds per row. Mean $\pm$ standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity ($\approx$2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs. 48
6.7 Validation protocols used for the selected GRU+Attention configuration on WLASL30. Source: self-created from k-fold and multi-seed outputs. 49
6.8 Aggregate WLASL30 validation results. K-fold artefacts store Top-1 and Top-5; Macro-F1 and balanced accuracy are therefore reported only for the fixed-split multi-seed protocol. Source: self-created from stored experiment outputs. 49

6.9 Per-seed test results of the selected GRU+Attention configuration on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs. . . 49
6.10 Configuration of the MS-ASL50 cross-dataset validation. Source: self-created from MS-ASL50 configuration files and experiment manifests. 50
6.11 MS-ASL reconstruction and MS-ASL50 subset audit. Source: self-created from the MS-ASL download audit and subset extraction report. 51
6.12 Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs. 51
6.13 Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs. 52
6.14 Configuration of the WLASL vocabulary-scaling experiment. Source: self-created from scaling manifests and result files. 53
6.15 Vocabulary scaling of the selected GRU+Attention configuration on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.8; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests. 53
6.16 Limitation categories, ordered from data acquisition constraints to modelling choices. Source: self-created from the experimental audit. 55
6.17 Usability questionnaire results for the SignToText demo. Means are computed on a 1 to 5 Likert scale. Source: self-created from Google Form responses. 57
6.18 Most valued SignToText features according to the usability questionnaire. Participants could select multiple options. Source: self-created from Google Form responses. 57

7.1 Ethical evaluation metrics based on Google's AI Principles [35] 64

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASL	American Sign Language
AWS	Amazon Web Services
BBox	Bounding Box
BiGRU	Bidirectional Gated Recurrent Unit
BPTT	Backpropagation Through Time
CEC	Constant Error Carousel
CNN	Convolutional Neural Network
CRISP-DM	Cross-Industry Standard Process For Data Mining
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
CV	Cross-Validation
FDP	Final Degree Project
FR	Functional Requirement
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection Over Union
JSON	JavaScript Object Notation
JSONL	JavaScript Object Notation Lines
LTS	Long-Term Support
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
MNIST	Modified National Institute Of Standards And Technology

Acronym	Meaning
MS-ASL	Microsoft American Sign Language
NFR	Non-Functional Requirement
OpenCV	Open Source Computer Vision Library
REST	Representational State Transfer
RGB	Red Green Blue
RNN	Recurrent Neural Network
RQ	Research Question
SLR	Sign Language Recognition
SO	Specific Objective
SSDLite	Single Shot Detector Lite
TTA	Test-Time Augmentation
UFV	Universidad Francisco De Vitoria
URL	Uniform Resource Locator
VRAM	Video Random-Access Memory
WLASL	Word-Level American Sign Language
XML	Extensible Markup Language
YAML	Human-Readable Data Serialisation Language
YOLO	You Only Look Once

Chapter 1

Introduction

Sign Language Recognition (SLR) represents a highly interdisciplinary paradigm at the intersection of digital accessibility, computer vision, and temporal sequence modelling, focused on capturing fully-fledged, sophisticated natural languages where semantic meaning is conveyed simultaneously rather than sequentially through distinct grammatical, syntactic, and visual-spatial structures.

1.1 Problem description

This project studies American Sign Language (ASL) recognition as an accessibility-oriented research problem. Unlike static image classification, sign recognition is inherently temporal: a single frame may show the handshape and approximate location of a sign, but it cannot represent the direction of movement, the order of articulatory changes or the rhythm of execution. The sign must therefore be interpreted from the ordered relationship between frames.

This temporal nature makes the problem substantially different from conventional object or gesture recognition. Two signs may share a similar hand configuration in an individual frame, while differing in trajectory, speed, orientation changes or relative hand position over time. In addition, real videos introduce variability in signer appearance, camera viewpoint, lighting conditions, signing style and execution speed. A recognition system must therefore learn not only visual patterns, but also how those patterns evolve across a short sequence.

The task addressed in this work is isolated word recognition. The system receives a short video clip and must assign it to a word gloss, that is, a written label used to annotate a sign. This formulation is more limited than continuous sign language translation, where complete sentences, grammar, coarticulation and broader linguistic context must be considered. However, isolated recognition still captures a central challenge of SLR: identifying a meaningful visual unit from motion and hand articulation.

1.2 Research scope and approach

The central research question of this work is whether compact visual representations can provide enough information for temporal models to classify short ASL video clips into word glosses. The

project evaluates different neural architectures under reproducible experimental conditions, with the objective of identifying a configuration that balances accuracy and computational cost. The work does not aim to replace professional interpreters or to deliver a production-ready translator.

The experimental work is grounded on public ASL datasets, mainly WLASL [1] and MS-ASL [2]. A mobile and server-based runtime prototype is also developed as a proof of concept, although the main scientific contribution is the experimental evaluation of the recognition approach rather than the deployment of an assistive product.

1.3 Outline

The remainder of the work is organised as follows:

- Chapter 2 reviews the State of the Art, providing the theoretical foundation in computer vision and deep learning required for this work. It examines convolutional neural networks and pose estimation, specifically MediaPipe Hands, before addressing temporal modelling through recurrent networks, Long Short-Term Memory (LSTM) layers, Gated Recurrent Unit (GRU) layers, Transformers and attention mechanisms. The chapter also evaluates previous research in SLR to establish the context for the current approach.
- Chapter 3 defines the main objective, the specific objectives, and the validation methods used to determine whether each objective has been achieved.
- Chapter 4 outlines the methodology and project work plan. It details the iterative, experiment-driven approach adopted throughout development and delineates the main project phases. Finally, it addresses the technical resources and constraints that influenced the scope and design of the project.
- Chapter 5 presents the technical development and system architecture. It provides a rigorous account of the data preprocessing pipeline and the experimental framework established for training and evaluation. The chapter concludes with a description of a functional proof of concept, demonstrating a real-time implementation through a mobile app and a FastAPI-based inference server.
- Chapter 6 reports the experimental evaluation. It includes an architecture comparison and sequence-length analysis, followed by validation on the WLASL30 and MS-ASL50 datasets. The chapter also addresses cross-dataset validation, vocabulary scaling and the qualitative validation design, and concludes with a critical discussion of the main limitations observed during experimentation.
- Chapter 7 analyses the ethical implications and social impact of the project, structured around the Google Artificial Intelligence Principles and ethical framework. It provides a critical examination of accessibility, privacy and algorithmic bias, while addressing the requirements for responsible deployment and the importance of reproducibility. Finally, it evaluates the broader social value of assistive technologies through these ethical guidelines.
- Chapter 8 reflects on the academic journey at Universidad Francisco de Vitoria and relates the project to the competences developed during the degree.
- Chapter 9 presents the conclusions and future work, summarising the main findings, the limitations of the current prototype, and possible directions for extending the research.

Chapter 2

State of the Art

This chapter reviews the technical foundations that support this project. It begins by exploring the evolution of Artificial Intelligence (AI), tracing the shift from early symbolic AI based on rules and logic techniques to modern sub-symbolic AI based on learning techniques. It then introduces the core components of Deep Neural Networks (DNNs) that drive this sub-symbolic paradigm, including feature extraction architectures, sequence and parallel processing architectures, and attention mechanisms. Finally, the chapter contextualises these technologies by reviewing previous work in Sign Language Recognition (SLR) and the main datasets used in American Sign Language (ASL) research.

2.1 From Symbolic to Subsymbolic AI

Early AI systems were symbolic, representing knowledge through explicit logic and predefined rule sets. They solved problems by applying human-authored rules, often organised as conditional decision trees. While this paradigm flourished within constrained environments, it proved inadequate for perception tasks, such as computer vision and speech, due to the inherent noise, ambiguity, and high-dimensionality of real-world data. This phenomenon, termed the "knowledge acquisition bottleneck", implies that as task complexity scales, the maintenance of handcrafted rule bases becomes untenable.

In the context of sign language, symbolic approaches could encode basic gesture prototypes, but they failed to capture the nuanced articulatory variation found across signers, camera angles and execution speeds. A fixed rule for a hand shape may describe an idealised sign, yet the observed motion in video changes with signer anatomy, partial occlusion, lighting and the temporal rhythm of the execution. These sources of variation make exhaustive symbolic description impractical.

Subsymbolic AI addresses the same class of problems through statistical learning. Instead of hard-coding expert rules for every possible case, a learning system estimates parameters from examples. This transition was driven by advances in neural networks and statistical learning theory, culminating in deep learning methods that learn hierarchical features directly from raw inputs [3]. By treating perception as a high-dimensional optimisation problem, these systems reduce the noise and ambiguity that previously led to the knowledge acquisition bottleneck.

In the SLR domain, this transition mirrors the evolution of the field: moving away from attempts to define a perfect hand-shape rules and towards architectures that extract latent features from raw video data. Consequently, the burden of feature engineering shifts from the human expert to the learning algorithm, allowing for a more fluid and robust interpretation of manual dynamics that aligns with the success seen in modern speech recognition and object detection frameworks.

2.2 Deep Neural Networks

A Deep Neural Network (DNN) is a parameterised function composed of layered computational structures designed to map inputs to specific outputs. Each layer receives an input vector, applies a linear transformation and passes the result through a non-linear activation function. A DNN stacks several such layers sequentially, allowing the model to compose simple, low-level mathematical transformations into progressively more abstract, high-level representations. Figure 2.1 shows this structure as a mapping from input features to an output prediction through hidden layers.

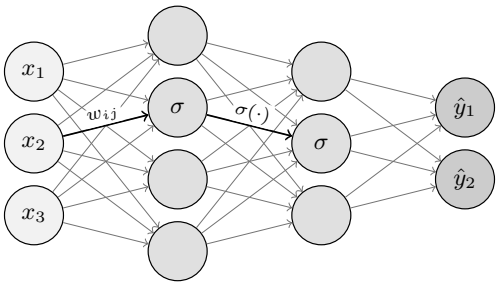


Figure 2.1: Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created.

For a layer l, the computation can be written as:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{W}^{(l)}$ contains the learned weights, $\mathbf{b}^{(l)}$ is the bias vector and σ is the activation function. The Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$, is a common activation because it introduces non-linearity while keeping the computation simple. The output layer depends on the task: classification layers estimate class probabilities, whereas regression layers estimate continuous quantities such as coordinates.

Training adjusts the parameters $\theta = \{\mathbf{W}, \mathbf{b}\}$ so that the function f_θ maps inputs to the desired targets. The model computes a prediction, compares it with the ground truth through a loss function and updates the parameters by gradient-based optimisation. In this sense, a DNN is not a list of explicit rules but an optimised function approximator.

This formulation explains the link between DNNs and feature extraction. Intermediate layers do not only transform the data numerically; they also produce representations that make the target task easier. In Computer Vision, these representations may correspond to edges, textures, object parts or

pose coordinates. Feature extraction therefore denotes the process of converting raw observations into informative variables that a later classifier or sequence model can use.

2.3 Feature Extraction Architectures

Feature extraction architectures convert video frames into representations that preserve relevant visual information while reducing irrelevant variation. In SLR, two families are especially important. Pixel-based techniques operate directly on image tensors, where each pixel stores colour or intensity values. Pose estimation techniques first detect anatomical points and then represent the signer through coordinates. The distinction matters because it determines whether the model learns from appearance information or from a geometric abstraction of the body and hands.

2.3.1 Pixel-Based Techniques

A digital image can be represented as a tensor $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H and W are height and width and C is the number of colour channels, as shown in Figure 2.2. Pixel-based techniques use this tensor as the direct input to the model. This preserves all visual information, including hand texture, background, illumination and camera artefacts.

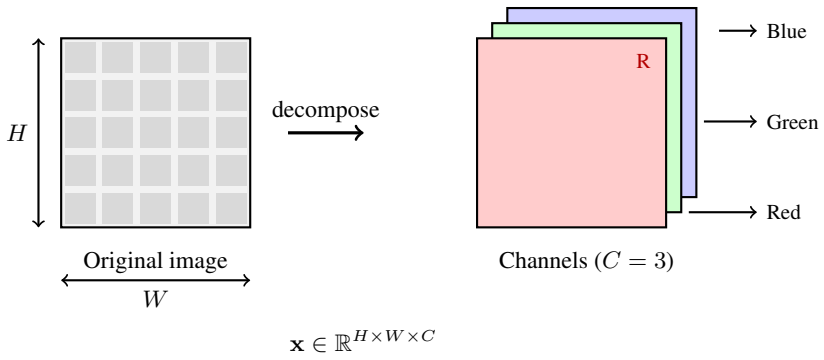


Figure 2.2: Decomposition of a digital RGB image into its three colour channels. The image is represented as a tensor $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H and W are the spatial dimensions and $C = 3$ corresponds to the red, green and blue channels.[1]

[1] Figure source code generated by OpenAI GPT-5.5 in High mode, OpenAI, May 20, 2026, from a prompt asking for a diagram representing a digital image as a tensor $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ and decomposing it into its RGB colour channels.

Convolutional Neural Networks (CNNs) are the standard DNN architecture for pixel-based computer vision [3]. CNN architectures stack convolutional, activation and pooling layers. A convolutional layer applies a small matrix of learned parameters, called a kernel or filter, across local regions of the image. At each spatial position, the kernel computes a response that indicates whether a visual pattern is present. The collection of responses produced by one kernel is a feature map.

For input tensor $\mathbf{x}$ and output feature map $\mathbf{y}$, a convolutional layer can be written as:

$$\mathbf{y}_{i,j,k} = \sigma\left(\sum_{c=1}^{C}\sum_{u,v}\mathbf{w}_{u,v,c,k}\mathbf{x}_{i+u,j+v,c} + b_k\right) \tag{2.2}$$

where $\mathbf{w}$ is the set of kernels, b_k is the bias for feature map k and σ is the activation function. Because the same kernel is reused across the image, convolutional layers encode translation equivariance: if a pattern shifts in the image, its corresponding activation shifts in the feature map.

Pooling layers reduce the spatial resolution of feature maps by summarising local neighbourhoods, commonly through maximum or average values. This supports local translation invariance, since small shifts in the input produce similar pooled activations. Early layers usually respond to oriented edges and simple textures; deeper layers combine those responses into larger shapes and object-level cues, as illustrated in Figure 2.3. AlexNet [4] and ResNet [5] are established examples of this hierarchical design.

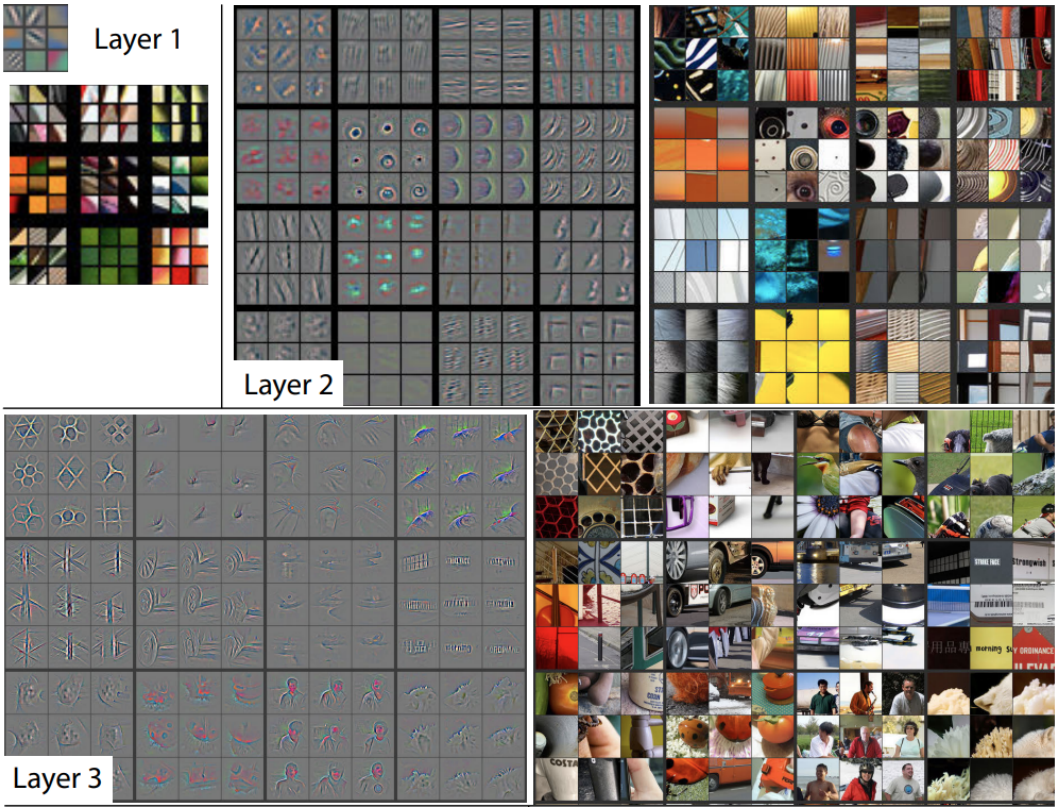


Figure 2.3: Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [6].

Figure 2.4 shows the AlexNet architecture. The diagram is relevant here because it illustrates the repeated pattern of convolution, pooling and fully connected layers that made deep pixel-based recognition practical.

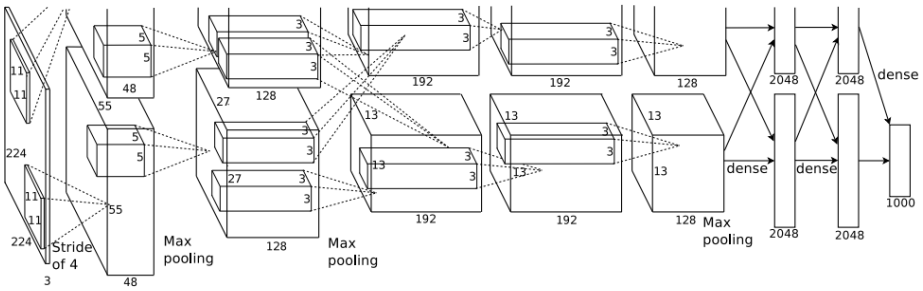


Figure 2.4: AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [4].

In SLR context, pixel-based CNNs can learn discriminative visual patterns directly from raw video frames. However, their main limitation is that they also receive information that may not define the sign, including background, skin tone and recording conditions. When the available dataset is limited, this can encourage the model to fit appearance correlations instead of sign structure.

2.3.2 Pose Estimation Techniques

Pose estimation involves predicting the spatial coordinates of anatomical points (landmarks) within an image or video frame, which constitutes a regression task because it requires calculating continuous numerical values rather than selecting a distinct class from a fixed set. In hand pose estimation, many landmarks correspond to joints, which are anatomical articulations such as knuckles, while fingertip landmarks mark the end of each finger. A regression-based pose estimator therefore predicts landmark coordinates rather than a discrete class label.

Figure 2.5 shows an example of the type of hand skeleton produced by a landmark-based hand tracking system. The image illustrates how the visual hand region is converted into a set of keypoints connected according to the anatomical structure of the hand.

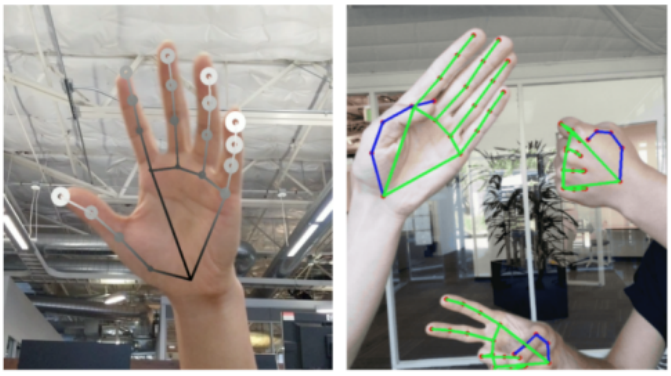


Figure 2.5: Rendered hand tracking result with hand landmarks and multi-hand tracking output. Source: Zhang et al. [7].

Pose pipelines are commonly organised as top-down or bottom-up systems. A top-down pipeline first detects the relevant object or body part, then estimates landmarks inside the detected region. A bottom-up pipeline first predicts candidate landmarks across the image, then groups them into instances. Both approaches convert raw image evidence into a structured geometric representation.

MediaPipe Hands [7] is a widely used hand pose estimation system. It uses the top-down structure shown in Figure 2.6: a hand detector proposes a region of interest, and a landmark model estimates hand coordinates inside that cropped region.

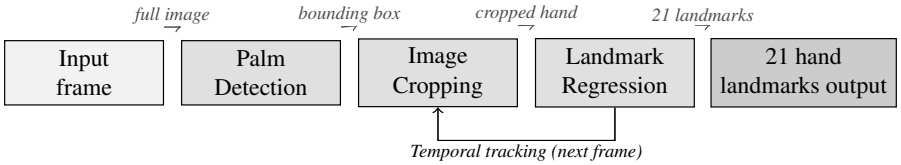


Figure 2.6: MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [7].

The process consists of the following stages:

1. **Palm detection:** The full image frame is processed by a single-shot detector, which estimates a bounding region around the palm and defines the hand region of interest.
2. **Image cropping:** The detected region is cropped and aligned around the hand, producing a normalised hand image for the landmark model.
3. **Landmark regression:** The landmark model receives the cropped hand image and regresses the coordinates of 21 three-dimensional hand landmarks.
4. **Temporal tracking:** When the hand remains visible, the pipeline reuses landmarks and region-of-interest information from the previous frame to obtain a continuous landmark sequence.

For SLR, landmark sequences provide a compact description of hand shape and movement. They remove much of the raw appearance signal and make it possible to derive additional geometric features such as bone vectors, joint angles and distances between fingers. The cost of this abstraction is that pose estimators may fail under occlusion, motion blur or unusual camera angles, and hand-only landmarks omit non-manual cues such as facial expression and upper-body movement.

2.4 Sequence Processing Architectures

Isolated sign recognition is a temporal classification problem. This means that the input is not a single image but an ordered sequence of feature vectors, one per frame or sampled time step. A sequence processing layer therefore receives $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$ and produces a representation that accounts for temporal order.

2.4.1 Recurrent Layers

Recurrent Neural Networks (RNNs) process a sequence by maintaining a hidden state. At time t, the layer combines the current input $\mathbf{x}_t$ with the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{2.3}$$

where ϕ is an activation function. The same parameters are shared at every time step, as shown in Figure 2.7. This sharing lets the layer process sequences of different lengths while applying the same transition rule throughout the video.

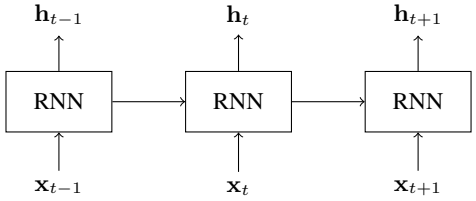


Figure 2.7: Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created.

Training recurrent layers uses Backpropagation Through Time (BPTT) [8], which unfolds the recurrent computation across the sequence and propagates the loss backwards through the unfolded graph. As the number of time steps increases, standard recurrent layers frequently encounter the vanishing gradient problem [9]. The gradient signal becomes too small to preserve information from early time steps, which limits long-range temporal modelling.

The recurrent layer described above represents the baseline architecture. To address its operational limitations, more sophisticated gated variants have been developed:

- Long Short-Term Memory (LSTM) layers, which introduce a cell state and gates to regulate information flow.
- Gated Recurrent Unit (GRU) layers, which simplify the LSTM design while retaining gated memory.

2.4.2 Long Short-Term Memory Layers

Long Short-Term Memory (LSTM) layers were proposed by Hochreiter and Schmidhuber [10] to improve the modelling of long temporal dependencies. An LSTM layer maintains two vectors: the hidden state $\mathbf{h}_t$, which is exposed to the next layer or output, and the cell state $\mathbf{C}_t$, which carries memory along the sequence.

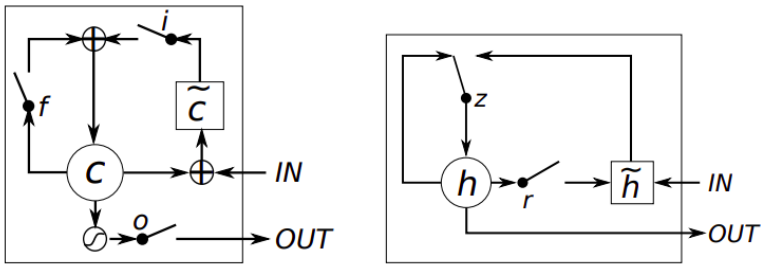
The LSTM cell regulates this memory through gates. The forget gate controls which information is removed from the cell state, the input gate controls which new information is written and the output gate controls which part of the cell state becomes the hidden state. Figure 2.8 (a) represents these gates as f, i and o, and shows how the cell state C provides a direct path for information flow.

This gated memory path is the mechanism referred to as the Constant Error Carousel. It reduces the decay of the error signal during BPTT and allows the layer to retain information over longer time

spans than a simple recurrent layer. The cost is computational: each time step requires several linear transformations, increasing both parameter count and training time.

2.4.3 Gated Recurrent Unit Layers

Gated Recurrent Unit (GRU) layers were introduced by Chung et al. [11] as a simpler gated recurrent alternative. A GRU layer removes the separate cell state and stores memory directly in the hidden state $\mathbf{h}_t$. It uses two gates, shown in Figure 2.8(b): the update gate z and the reset gate r.



(a) Long Short-Term Memory (b) Gated Recurrent Unit

Figure 2.8: Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11].

The GRU gate equations at time t are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \tag{2.4}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \tag{2.5}$$

The reset gate controls how much previous information is used to compute the candidate hidden state $\tilde{\mathbf{h}}_t$. The update gate controls the interpolation between the previous hidden state and the candidate state:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{2.6}$$

When $\mathbf{z}_t$ is close to zero, the layer copies most of the previous hidden state. When it is close to one, the layer replaces it with new information. This gives GRU layers a compact mechanism for modelling temporal dependencies with fewer parameters than LSTM layers.

2.5 Attention Mechanisms

Attention mechanisms assign different weights to different parts of an input sequence. In neural machine translation, Bahdanau et al. [12] introduced attention to avoid compressing a full source sentence into a single fixed-length vector. The same idea applies to video recognition: not every frame contributes equally to the final prediction.

Given encoder states $\{\mathbf{h}_t\}_{t=1}^{T}$, an attention module computes a score e_t for each time step and normalises the scores with a softmax function:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}, \quad e_t = a(\mathbf{s}, \mathbf{h}_t) \tag{2.7}$$

where $a(\cdot)$ is a scoring function and α_t is the attention weight assigned to time step t. The context vector is then computed as:

$$\mathbf{c} = \sum_{t=1}^{T} \alpha_t \mathbf{h}_t \tag{2.8}$$

Figure 2.9 shows this weighted combination: the decoder state attends to encoder annotations through weights $\alpha_{t,1}, \alpha_{t,2}, \ldots, \alpha_{t,T}$.

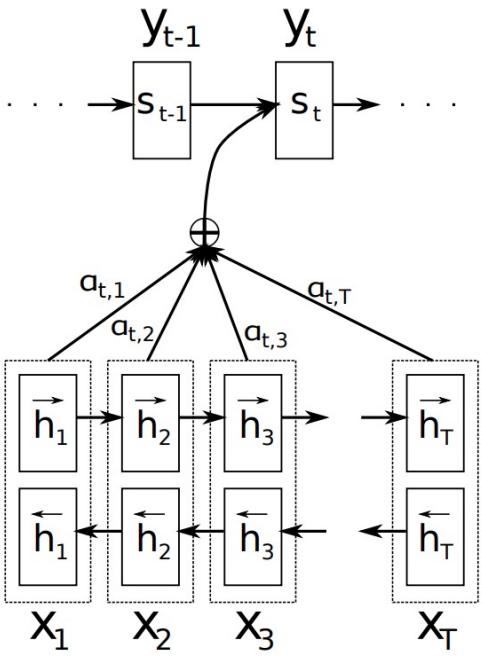


Figure 2.9: Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [12].

For example, in an SLR clip of the sign *drink*, the first frames may show the hand moving towards the mouth, the central frames may contain the class-defining motion and the final frames may show the hand returning to rest. Temporal attention can assign higher weights to the central frames and lower weights to the preparation and recovery frames. This does not remove the need for a sequence model; it adds a learned weighting stage over the temporal representation produced by that model.

2.5.1 Transformer Layers

Transformer layers extend the attention idea by making self-attention the main operation of the sequence model [13]. Instead of updating a hidden state step by step, as in a recurrent layer, self-attention compares every position in the sequence with every other position. Each time step is projected into query, key and value vectors. The query-key similarities determine which positions

are most relevant to each other, and the value vectors are combined according to those attention weights.

This formulation allows a Transformer to model relationships between distant frames without passing information through all intermediate time steps. It also supports parallel computation across the sequence, which is useful when large datasets and long inputs are available. However, because the model does not process frames in their natural order by default, positional information must be added so that the network can distinguish earlier frames from later ones.

In SLR, Transformer-based models are relevant because signs often contain dependencies between non-adjacent moments of the motion. At the same time, they can be more data-demanding than recurrent layers, especially in small word-level datasets. For this reason, Transformers are best understood in this work as attention-based sequence models to compare against recurrent alternatives, rather than as an automatic replacement for LSTM or GRU layers.

2.6 Related Work in Sign Language Recognition

SLR research can be organised according to the recognition setting and the type of visual representation used by the model. Survey work distinguishes between isolated sign recognition, continuous sign recognition and sign language translation [14], [15], [16]. Isolated recognition assigns a single gloss to a trimmed clip, which is the scope of this project. Continuous recognition must segment and classify signs from longer streams, while translation must produce spoken-language text and account for grammar, context and non-manual markers.

Detection-based approaches: Earlier vision pipelines often treated hands as regions to be detected before classification. Modern object detectors such as Faster Region-based Convolutional Neural Network (Faster R-CNN) [17], Single Shot MultiBox Detector (SSD) [18], EfficientDet [19] and the You Only Look Once (YOLO) family [20], [21] frame this problem as locating objects and assigning class scores in images. YOLO is especially relevant for real-time hand detection because it performs detection in a single forward pass, and YOLOv8 is a current implementation frequently used in practical detection pipelines [22].

Pixel-based deep learning approaches: With the adoption of DNNs, many SLR systems moved from handcrafted descriptors towards models that learn visual features directly from video frames. Pixel-based CNNs extract frame-level appearance patterns, while recurrent layers model their temporal evolution and attention modules weight the most informative moments of the sequence. Transformer-based work such as Sign Language Transformers [23] and STMC-Transformer [24] shows how attention-based sequence models have been applied to continuous recognition and translation settings.

Pose-based approaches: Pose-based methods replace raw frames with landmark sequences, reducing dimensionality and limiting the influence of background appearance. Bohacek and Hruz [25] proposed a pose-based Transformer for word-level SLR, while Sign2Pose [26] combines pose features and Transformer-based gloss prediction on WLASL. This project is situated in the pose-based line: MediaPipe Hands provides hand landmarks, and the recognition model evaluates recurrent and attention-based temporal processing.

Datasets and evaluation context: Word-level ASL recognition is strongly associated with WLASL and MS-ASL. Li et al. [1] introduced WLASL together with benchmark comparisons for word-level recognition from video, while Joze and Koller [2] introduced MS-ASL as a larger ASL benchmark. Both works helped move isolated SLR evaluation away from small curated vocabularies and towards larger, noisier video collections. Large-scale work such as BSL-1K [27] also highlights that dataset scale, weak supervision and cross-dataset transfer remain central concerns in sign recognition research.

2.7 Dataset Limitations

SLR systems depend strongly on the datasets available for training and evaluation. Unlike general image recognition, sign language recognition still lacks abundant large-scale benchmarks across languages, domains and signer populations. Dataset design therefore shapes what a recognition model can learn: the vocabulary determines the classification space, recording conditions affect visual variation and annotation practice defines the gloss labels used for evaluation.

The main limitations are fourfold. First, many sign datasets contain a limited number of examples per class, which makes it difficult to train high-capacity temporal models without overfitting. Second, class imbalance is common: frequent signs tend to have many more examples than rare signs, so aggregate accuracy may hide poor behaviour on under-represented glosses. Third, clips collected from online sources introduce uncontrolled variation in camera quality, viewpoint, signer appearance, background, duration and execution style. Finally, dataset attrition can occur when source videos are removed or made private, reducing reproducibility over time.

These limitations affect how experimental results should be interpreted. A model can perform well on a constrained benchmark while still struggling to generalise to new signers, recording conditions or larger vocabularies. Auxiliary resources such as static alphabet datasets or hand-detection corpora are useful for engineering stages, but they should not be treated as word-level SLR benchmarks because they do not evaluate temporal sign dynamics.

Chapter 3

Objectives

This chapter defines the purpose of the project, decomposes it into specific and verifiable objectives and states how each objective is validated in the remainder of the document. The objectives describe the research and project outcomes pursued by the work. They are therefore different from the software requirements presented in Section 5.2, which specify the functional and non-functional behaviour expected from the implemented system.

3.1 Main objective

The main objective of this Final Degree Project (FDP) is to design, implement and experimentally validate a research prototype for word-level American Sign Language (ASL) recognition from short video clips through Computer Vision. The purpose is not to build a production-ready translator, but to study how convolutional and temporal recognition approaches can support feasible and reproducible isolated-word sign recognition.

The hypothesis derived from this objective is the following:

> *A compact sequence representation of sign movement can retain enough discriminative information to recognise a controlled vocabulary of isolated ASL signs while reducing dependence on raw video appearance.*

This hypothesis is tested by comparing candidate architectures, selecting the best-performing configuration and evaluating its behaviour across repeated runs, datasets and vocabulary sizes.

3.2 Specific objectives

The main objective is decomposed into six specific objectives. Each one is defined as a measurable project outcome and is later mapped to a validation method in Section 3.3.

- **SO1 - Analyse state-of-the-art techniques in Computer Vision and temporal modelling:** This objective covers the relevant foundations of computer vision, pose estimation, MediaPipe Hands, sequence modelling, convolutional architectures, recurrent architectures, atten-

tion mechanisms and Transformer architectures, with emphasis on their applicability to word-level sign language recognition.

- **SO2 - Define a landmark-based data representation for ASL video clips:** This objective focuses on transforming raw ASL video clips into normalised sequences of three-dimensional hand landmarks suitable for temporal modelling, while documenting the datasets, preprocessing decisions and limitations introduced by landmark extraction. The resulting representation is intended to provide a compact and consistent description of hand movement and spatial relationships across frames, facilitating the development of sign language recognition models while reducing sensitivity to background variation, lighting conditions and camera position.
- **SO3 - Design and compare candidate convolutional and temporal architectures for word-level ASL recognition:** This objective evaluates alternative architectures under a controlled 30-class subset of the WLASL dataset, referred to as WLASL30, and identifies the configuration most suitable for the available data regime. It also examines each architecture's ability to capture temporal dependencies and dynamic hand movements from landmark sequences, while assessing performance, generalisation, computational cost and consistency across different sign categories. The objective further analyses the trade-off between model complexity and recognition accuracy in order to determine the most appropriate architecture for reliable word-level ASL recognition.
- **SO4 - Evaluate the selected recognition pipeline through reproducible experimental protocols:** This objective evaluates the selected model using standard classification metrics and protocols that reduce dependence on a single random seed or split, including multi-seed evaluation and stratified cross-validation. It also assesses the stability, consistency and generalisation capability of the recognition pipeline across different evaluation settings to ensure that the reported performance is reliable and reproducible.
- **SO5 - Analyse the generalisation and scalability of the recognition pipeline:** This objective tests whether the selected approach transfers effectively to the 50-class subset of the independent MS-ASL dataset, referred to as MS-ASL50, and analyses how performance changes as the WLASL vocabulary size increases. It also evaluates the stability and scalability of the recognition pipeline under more challenging classification settings, identifying potential limitations related to vocabulary expansion, dataset variability and model generalisation across different signing conditions.
- **SO6 - Evaluate the runtime proof of concept and the ethical implications of the recognition approach:** This objective integrates the recognition pipeline into an end-to-end mobile/server prototype as a proof of concept and analyses its accessibility motivation, privacy implications, bias risks, reproducibility constraints and responsible-deployment requirements.

3.3 Validation methods

Table 3.1 maps each specific objective to the method used to validate it and to the document evidence where the committee can verify the result. Detailed numerical results are intentionally left to the Experiments chapter; this section only defines how objective achievement is assessed.

The experimental objectives use four classification metrics. Let N be the number of test clips, C the number of classes, y_i the ground-truth label of clip i, $\hat{y}_i$ the highest-scoring predicted label and TopK_i the set of the K highest-scoring predicted labels for clip i.

- **Top-1 accuracy -** This metric measures the proportion of test samples for which the highest-scoring prediction matches the ground-truth label:

$$\text{Top1} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}(\hat{y}_i = y_i) \tag{3.1}$$

where $\mathbb{1}(\cdot)$ is 1 when the condition is true and 0 otherwise. For example, if 20 out of 34 WLASL30 test clips are classified correctly as the first prediction, then $\text{Top1} = 20/34 = 58.82\%$.

- **Top-5 accuracy -** This metric measures whether the ground-truth label appears among the five highest-scoring predictions:

$$\text{Top5} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}(y_i \in \text{Top5}_i) \tag{3.2}$$

For example, if the correct ASL sign appears in the five proposed labels for 27 out of 34 clips, then $\text{Top5} = 27/34 = 79.41\%$.

- **Macro-F1 -** This metric evaluates class-level behaviour by computing the F1 score independently for each class and then averaging all classes with equal weight:

$$\text{MacroF1} = \frac{1}{C}\sum_{c=1}^{C} \text{F1}_c \tag{3.3}$$

The F1 score for class c combines precision and recall:

$$\text{F1}_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{3.4}$$

For example, a class with precision 0.80 and recall 0.60 obtains $\text{F1} = 2 \cdot (0.80 \cdot 0.60)/(0.80 + 0.60) = 0.69$. Macro-F1 is useful when frequent classes could otherwise dominate the average.

- **Balanced accuracy -** This metric averages recall across classes:

$$\text{BalancedAccuracy} = \frac{1}{C}\sum_{c=1}^{C} \text{Recall}_c \tag{3.5}$$

For example, if three classes obtain recall values of 1.00, 0.50 and 0.00, then $\text{BalancedAccuracy} = (1.00 + 0.50 + 0.00)/3 = 0.50$.

The evaluation protocols complement these metrics. A **multi-seed evaluation** trains the same configuration with different random seeds and reports the mean and standard deviation, which helps separate model behaviour from random initialisation effects. **Stratified cross-validation** splits the dataset into folds while preserving class proportions, so that the evaluation does not depend on a single train-test partition.

Table 3.1 indicates, for each specific objective, which of the metrics defined above are used and where the corresponding evidence is presented in the document.

Table 3.1: Validation mapping for the specific objectives.

Objective	Validation method and metrics	Evidence in the document
SO1	The objective is validated by presenting a structured review of the main techniques needed to understand the proposed system, including convolutional architectures, pose estimation, MediaPipe Hands, recurrent layers, attention mechanisms and Transformer architectures.	State of the Art chapter (Chapter 2) and the cited literature used to justify the methodological choices.
SO2	The objective is validated by documenting the pipeline that converts ASL videos into normalised landmark sequences and by describing the datasets, extraction process, preprocessing steps and data-quality constraints.	System design and dataset description in Section 5.1 and Section 5.3; methodological pivot in Section 6.2.
SO3	The objective is validated by comparing candidate convolutional and temporal architectures on the same WLASL30 benchmark using Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy, followed by selection of the best-performing configuration under the controlled comparison.	Architecture selection in Section 6.3 and Table 6.3.
SO4	The objective is validated through the multi-seed protocol, the sequence-length sweep, stratified 5-Fold Cross-Validation and the reporting of Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy across repeated runs.	Experimental methodology in Section 6.1; sequence-length sweep in Table 6.6; WLASL30 validation in Section 6.5 and Table 6.9.
SO5	The objective is validated by retraining and evaluating the same pipeline on MS-ASL50 without architectural changes and by measuring Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy across progressively larger WLASL vocabularies.	Cross-dataset validation in Section 6.6, Table 6.12 and Table 6.13; vocabulary scaling in Section 6.7 and Table 6.15.
SO6	The objective is validated by describing the runtime prototype architecture, defining a qualitative validation protocol for live use and evaluating the ethical and social implications of the system as a research prototype.	Runtime architecture in Section 5.1; qualitative protocol in Section 6.9; ethical and social-impact analysis in Chapter 7.

Chapter 4

Methodology and Work Plan Overview

This chapter presents the methodology followed throughout the project, outlining the resources and tools employed, the technical approaches adopted at different stages, the workflow used to guide development and the sequence of activities undertaken. It also discusses the practical and technical constraints that influenced the decisions made during implementation. The technical development of the system is described in Section 5.1 and following, while the experimental evidence is reported in Section 6.1 and following.

4.1 Materials and Resources

The project required three resource categories: (1) Hardware; (2) Software; and (3) Datasets. Each category is described below and cross-referenced to the chapter section where the technical detail is reported.

4.1.1 Hardware

All training, evaluation and landmark extraction were performed on a single local workstation whose specification is the operational environment of Section 5.1.5. The full table of components and version constraints is Table 5.1 and is not repeated here. The relevant hardware resource for the work plan is the GPU, an NVIDIA RTX 3050 Ti Laptop with 4 GB of VRAM. The practical implications of this memory ceiling are discussed in Section 4.5.

4.1.2 Software

The project combines a Python-based training pipeline, a FastAPI inference server and a React Native mobile client. Table 4.1 summarises the most relevant software libraries and tools used during development. The exact version constraints are locked in `pyproject.toml`, `04_server/requirements.txt` and `03_app/signtotext-expo/package.json`, allowing the operational environment to be reconstructed from a clean clone of the repository.

Table 4.1: Main software libraries and tools used in the project.

Library or tool	**Version**	**Use in the project**
Python	3.10+	Main programming language for preprocessing, landmark extraction, model training and evaluation scripts.
PyTorch	2.0+	Definition, training and evaluation of the neural sequence models used for word-level ASL recognition.
CUDA	12.x	GPU acceleration for model training and inference experiments on the local workstation.
MediaPipe Hands	0.10+	Extraction of 21 three-dimensional hand landmarks per frame from ASL video clips.
OpenCV	4.8+	Video decoding, frame extraction and image handling during preprocessing and inference.
NumPy	1.24+	Storage and manipulation of landmark arrays and numerical features.
scikit-learn	1.2+	Dataset splitting, classification metrics and experimental evaluation utilities.
Matplotlib and Seaborn	3.7+ / 0.12+	Generation of plots, confusion matrices and visual summaries of experimental results.
TensorBoard	2.12+	Monitoring of training curves and experiment logs.
FastAPI	0.111	Implementation of the inference server exposed to the mobile client.
Uvicorn	0.29	ASGI server used to run the FastAPI application.
Pydantic	2.0+	Data validation and structured request/response models in the server.
React Native and Expo	0.81.5 / 54.x	Development of the mobile proof-of-concept application.
Expo Camera, File System, Image Picker and Speech	55.x / 14.x	Mobile recording, file access, media selection and speech output in the client.
Git and GitHub	–	Version control, repository hosting and development history management.
LaTeX	–	Writing and formatting of the final project report.

4.1.3 Datasets

The project used six datasets, grouped by their role in the development process.

1. **EgoHands:** EgoHands provides annotated hand images recorded from egocentric video. It was used during the initial hand-localisation stage to train and evaluate a detector capable of finding hands before any sign-recognition model was applied.
2. **Kaggle Hand Dataset:** The Kaggle Hand Dataset supplied additional hand images for the same localisation stage. It increased the visual variability of the detector training data and complemented EgoHands with examples from different capture conditions.
3. **Sign Language MNIST:** Sign Language MNIST contains static greyscale images of alphabet handshapes. It was used only in the exploratory letter-classification stage, where the objective was to test whether a simple image classifier could recognise isolated hand configurations.
4. **ASL Alphabet:** The ASL Alphabet dataset contains static colour images of American Sign Language (ASL) alphabet signs. It supported the same early letter-recognition branch as Sign Language MNIST, but it did not form part of the final word-level recognition pipeline because it does not contain temporal sign videos.
5. **WLASL:** The Word-Level American Sign Language (WLASL) dataset is the main video benchmark used for word-level ASL recognition. The project builds WLASL30 for controlled architecture comparison and also uses larger WLASL subsets to analyse vocabulary scaling.
6. **MS-ASL:** The Microsoft American Sign Language (MS-ASL) dataset is an independent word-level ASL video dataset reconstructed from public video sources. The project uses MS-ASL50 as a cross-dataset validation benchmark to test whether the selected recognition pipeline generalises beyond WLASL.

4.2 Methods

This section describes the methods used to carry out the work introduced in Chapter 1. It covers the theoretical basis, algorithms, metrics, exploratory case studies and replication support used in the project. Exact hardware specifications are reported in Section 5.1.5; exact experimental hyperparameters are reported in Section 6.1.

4.2.1 Theoretical and Algorithmic Basis

The final recognition method is based on supervised sequence classification. Each input sample is a short ASL video clip, and the output is a gloss label from a fixed vocabulary. The method combines pose estimation, recurrent sequence modelling and temporal attention.

Pose estimation provides the input representation. A landmark is a three-dimensional keypoint that represents a specific position on the detected hand, such as a fingertip or joint. MediaPipe Hands estimates 21 landmarks per detected hand, so each valid frame is represented as 63 scalar features. This converts the recognition problem from pixel-based video classification into classification over structured skeletal sequences.

The sequence model uses Gated Recurrent Unit (GRU) layers for temporal modelling. A bidirectional GRU (BiGRU) layer processes the landmark sequence in forward and reverse temporal order,

then combines both hidden-state streams. Training recurrent layers uses Backpropagation Through Time (BPTT), which unfolds the sequence across time steps so that the loss can update the recurrent weights. The selected architecture adds a temporal attention layer over the BiGRU outputs and uses the resulting context vector for classification. Throughout the document, this full architecture is referred to as the GRU+Attention model.

4.2.2 Recognition Pipeline

The implemented recognition pipeline has four stages:

1. **Video acquisition and decoding:** The input is a labelled video clip from WLASL, MS-ASL or the runtime prototype. OpenCV decodes the clip frame by frame. The output is an ordered frame sequence that preserves the temporal structure of the sign.
2. **Landmark extraction and normalisation:** MediaPipe Hands estimates 21 three-dimensional landmarks per frame. The coordinates are wrist-centred and length-normalised by the wrist-to-middle-finger distance. Frames where MediaPipe fails are linearly interpolated from neighbouring valid frames, and clips with a detection rate below 0.3 are discarded. The output is a per-clip `.npy` array of shape $(T, 21, 3)$ plus metadata.
3. **Training-time augmentation:** During training, the landmark coordinates are transformed through rotation, scaling, translation, additive Gaussian noise and horizontal mirroring. These operations simulate variation in signer position and execution while preserving the skeletal representation.
4. **Sequence classification:** The GRU+Attention architecture receives the landmark sequence and produces class scores over the selected gloss vocabulary. The same training loop also supports the alternative architectures compared in Section 6.3, so the comparison changes the sequence-modelling component while keeping the input representation fixed.

4.2.3 Training, Metrics and Validation

Training uses AdamW optimisation, cross-entropy loss with label smoothing, cosine learning-rate annealing and early stopping. The numerical values of these hyperparameters are reported in the experimental setup of Section 6.1.

The evaluation metrics are Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy. Top-1 accuracy measures whether the highest-scoring prediction matches the ground truth. Top-5 accuracy measures whether the ground truth appears within the five highest-scoring predictions. Macro-F1 averages per-class F1 scores without weighting by class frequency, while balanced accuracy averages per-class recall.

Validation uses Stratified 5-Fold Cross-Validation, multi-seed evaluation, sequence-length analysis and cross-dataset validation. Stratified 5-Fold Cross-Validation preserves the per-class distribution across folds. Multi-seed evaluation repeats the same configuration under different random seeds and reports mean and standard deviation. Sequence-length analysis tests the effect of the temporal input window. Cross-dataset validation retrains the same pipeline on MS-ASL50 to assess generalisation beyond WLASL.

4.2.4 Exploratory Case Studies

Two exploratory case studies preceded the final landmark-based method. The first used You Only Look Once version 8 nano (YOLOv8n) for hand localisation on EgoHands and the Kaggle Hand Dataset. The second used Convolutional Neural Network (CNN) classifiers for static ASL letter recognition on Sign Language MNIST and ASL Alphabet. These studies showed that hand localisation and isolated letter classification were tractable, but they did not address the temporal structure required for word-level ASL recognition. Their role in the final method is therefore diagnostic: they justify the shift from pixel-based letter recognition to landmark-based word recognition.

4.2.5 Proof-of-Concept Design and Expected Outcomes

The proof of concept connects the trained recognition model to a runtime application. The mobile client captures a short video clip and sends it to a FastAPI server. The server applies the same MediaPipe landmark extraction used during training, loads the selected checkpoint and returns Top-K predictions with confidence scores. The client can display the prediction shortlist and optionally convert the Top-1 label to speech.

The expected outcome of the proof of concept is not a deployment-ready translator. Its purpose is to demonstrate that the trained landmark-based recogniser can be integrated into an end-to-end capture, inference and response loop. The quantitative validity of the recognition model is therefore assessed in the Experiments chapter, while the prototype demonstrates the feasibility of using the selected model in an interactive setting.

4.2.6 Replication Support

Replication depends on controlled configuration and traceable experiment outputs. Dataset paths, model options and training settings are stored in YAML configuration files under `configs/`. Each training or evaluation run writes an `experiment_manifest.json` file with the git commit, resolved configuration, hardware identifier, wall-clock time and final metrics. The development environment is specified through `pyproject.toml` for the training package and `04_server/requirements.txt` for the inference server. The full reproduction procedure is documented in `docs/reproducibility_guide.md`.

4.3 Methodology

The project followed a hypothesis-driven iterative methodology, adapted from the Lean Startup framework [28] to the research context of this work. In a research setting, the build-measure-learn loop is reinterpreted as a hypothesis-experiment-learn loop: each iteration tests a technical or methodological hypothesis through a controlled experiment, measures the result against defined metrics, and revises the next iteration accordingly. The original commercial-validation step is replaced with empirical experimental validation, which is what differentiates this adaptation from the standard Lean Startup formulation. This approach was appropriate because the early project direction changed after the exploratory stage, and the work plan had to preserve room for evidence-driven correction. It also made the methodological process explicit, since every relevant decision was linked to an observed result rather than to an initial assumption.

Each iteration followed four stages:

1. **Hypothesis definition:** The input was a previous devlog entry, tutor comment, experimental result or implementation problem. The output was a testable technical hypothesis, together with the dataset, metric and configuration needed to evaluate it.
2. **Build:** The input was the hypothesis and the current repository state. The output was an implementation change, a YAML configuration file or both, committed to Git so that the experiment could be traced to a specific version of the code.
3. **Measure:** The input was the committed implementation and its configuration. The output was a set of experiment artefacts: metrics, checkpoints, plots and an `experiment_manifest.json` file.
4. **Learn:** The input was the measured evidence from the previous stage. The output was a documented decision to keep, revise or discard the hypothesis, recorded in the development log before the next iteration began.

The pivot recorded in Section 6.2 is the clearest application of this loop. The project began with a pixel-based route that combined hand localisation with static letter classification. The detector and the letter classifiers were functional on their own datasets, but the combined route showed that the central problem was not locating the hand. The central problem was representing the temporal structure of complete signs. The next iteration therefore replaced the pixel-based recognition route with the landmark-based word-recognition pipeline used in the rest of the project.

Tutor meetings acted as external review checkpoints within the iterative process. Examples include removing Test-Time Augmentation (TTA) from the final evaluation protocol, adopting Stratified 5-Fold Cross-Validation, extending validation to MS-ASL50 and adding multi-seed evaluation to reduce dependence on a single random initialisation.

Documentation was maintained as part of the methodology rather than as a final clean-up task. A structured development log was kept throughout the project, containing dated entries for each main work session and recording the hypothesis, change, result and next action of each iteration. The work plan in Section 4.4 is reconstructed from that log, the proposal, experiment artefacts and supervision notes.

4.4 Work Plan

The executed work plan was reconstructed from the original proposal, development notes, experiment artefacts and supervision notes. It is organised into seven phases (A to G): the first two correspond closely to the proposal plan, while the remaining phases reflect the methodological pivot from static, pixel-based recognition to landmark-based word-level ASL recognition.

The original work plan from the project proposal (January 2026) followed a sequential structure with separate phases for hand detection, static handshape recognition, dynamic sign recognition, temporal fusion, language generation, mobile prototype, integration and validation. The executed plan diverged after the methodological pivot described in Section 6.2, which replaced the static letter recognition route with landmark-based word recognition and folded the temporal fusion stage into the selected sequence model. Figure 4.1 summarises the executed work plan at the start of the section, showing the start and end of each phase and making the main deviation from the initial proposal visible: the shift from static letter recognition to landmark-based word-level recognition.

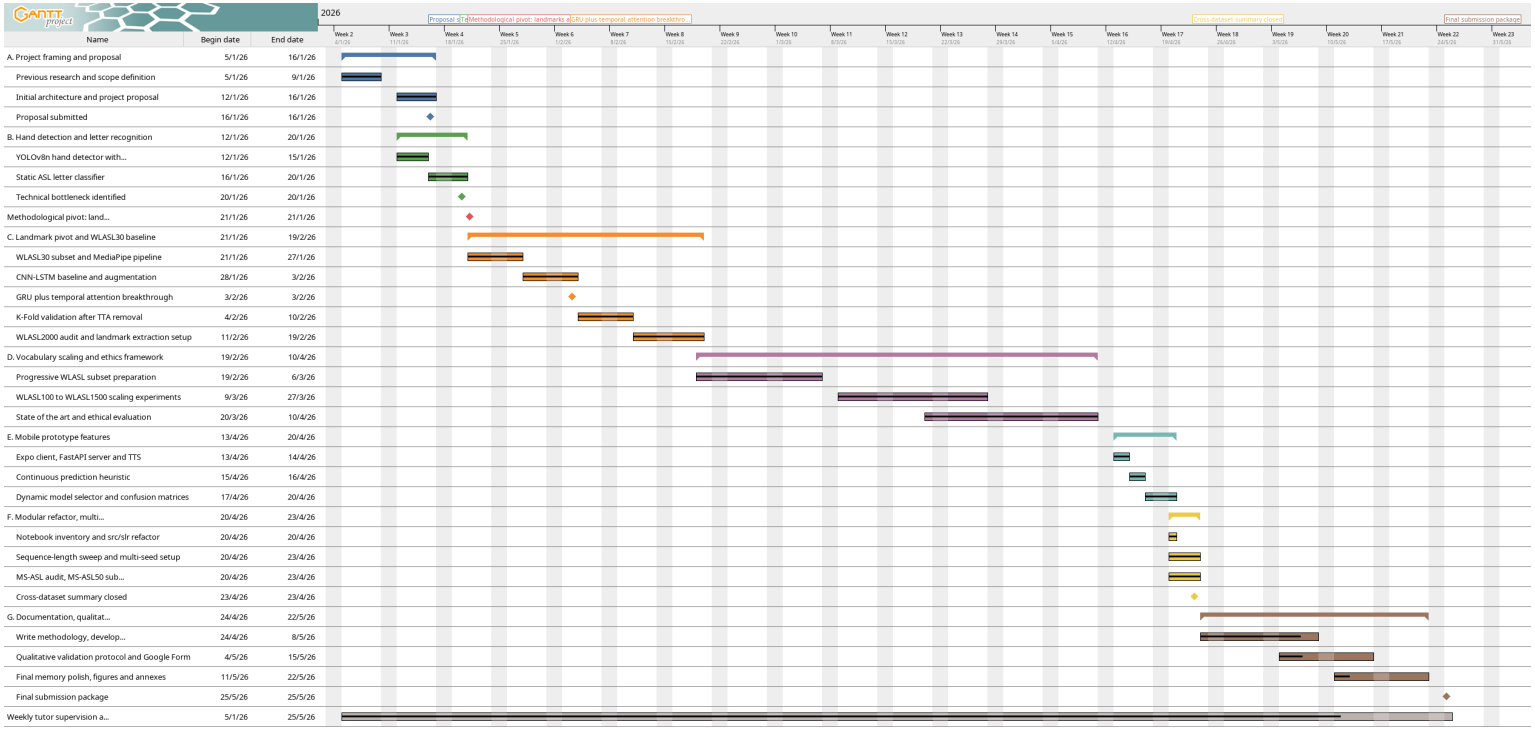


Figure 4.1: Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration and final documentation. Source: self-created with GanttProject.

1. **Phase A - Project framing and proposal (from 5 January 2026 to 16 January 2026):** The project formally started with problem definition, preliminary literature review, scope selection and proposal preparation. This phase defined the accessibility motivation, the initial system architecture, the first work schedule and the research direction. No model training was carried out at this stage. The main output was the approved project proposal and its initial planning material.
2. **Phase B - Hand detection and static letter recognition (from 12 January 2026 to 20 January 2026):** The initial technical work explored the proposal's first recognition route: localising hands in images and classifying static ASL letters. A YOLOv8n detector was trained on a combined EgoHands and Kaggle Hand Dataset corpus, and CNN-based classifiers were explored on static ASL alphabet datasets. These components reached working performance on their own tasks, but they also revealed a limitation in the original direction: the central research challenge was not hand localisation, but representing the temporal structure of complete signs.
3. **Phase C - Landmark pivot and WLASL30 baseline (from 21 January 2026 to 19 February 2026):** The project pivoted from pixel-based letter recognition to word-level recognition using MediaPipe hand landmarks. WLASL30 was built as the first controlled benchmark, and several sequence architectures were compared under the same landmark input representation. The GRU+Attention architecture became the best-performing configuration because it combined recurrent temporal modelling with an attention layer able to weight informative frames. After supervisory review, TTA was excluded from the final evaluation protocol, and the WLASL30 result was strengthened through Stratified 5-Fold Cross-Validation.
4. **Phase D - Vocabulary scaling and ethical framework (from 19 February 2026 to 10 April 2026):** Once the GRU+Attention model had been selected, the next phase tested how the approach behaved as the WLASL vocabulary increased beyond the initial 30-class setting. The model was retrained on progressively larger WLASL subsets to quantify the effect of vocabulary size and per-class data scarcity. In parallel, the ethical and social-impact chapter was developed, including the accessibility motivation, privacy implications of landmark-based representations, bias considerations, reproducibility and the responsible-deployment limits of the prototype.
5. **Phase E - Runtime prototype (from 13 April 2026 to 20 April 2026):** The technical work expanded from offline training to a runtime proof of concept. A mobile client and FastAPI server were integrated so that short clips could be captured, sent for inference and returned as Top-K word predictions. Additional prototype features included text-to-speech, model selection from the client and a heuristic continuous-prediction demo mode based on overlapping windows. This phase demonstrates end-to-end feasibility, but it is not treated as the main research result. The quantitative contribution remains the landmark-based recognition pipeline and its validation.
6. **Phase F - Refactoring and cross-dataset validation (from 20 April 2026 to 23 April 2026):** The notebook-based experimental code was consolidated into a modular Python package, with configuration-driven training, structured experiment manifests, reusable evaluation scripts and preserved checkpoints and plots. This refactor improved reproducibility and made it possible to rerun the same pipeline across datasets. MS-ASL50 was then added as an independent validation benchmark, and the final evaluation protocol incorporated multi-seed runs, a sequence-length analysis, cross-dataset validation and comparison against the WLASL results.
7. **Phase G - Documentation and qualitative validation (from 24 April 2026 to 25 May**

2026): The final phase focused on document writing, result consolidation, figure and table preparation and qualitative validation of the runtime prototype. The experimental chapter was organised into quantitative and qualitative tracks. The quantitative track reports completed model evaluations, while the qualitative track describes the participant protocol and reports the usability evidence collected from the participant sessions.

4.5 Constraints and Limitations

This section lists the different constraints that shaped the work plan and implementation. They are ordered by their impact on project scope and design. Limitations of the experimental conclusions themselves, including standard deviations, test-set granularity and single-architecture cross-dataset validation, are treated separately in Section 6.8.

1. **Computing infrastructure:** Every training and evaluation run executed on a single NVIDIA RTX 3050 Ti Laptop GPU with 4 GB of VRAM. This imposed a hard memory ceiling on the temporal window, batch size and model size that could be tested. The practical effect was to prioritise compact recurrent architectures and to exclude larger video models that would not fit reliably within the available hardware.
2. **Dataset reconstruction and landmark coverage:** Both word-recognition datasets depend on external video sources rather than on locally distributed video files. WLASL relies on URLs that have decayed since publication, and the MediaPipe detection rate on the surviving WLASL clips is approximately 69 %. MS-ASL is reconstructed from per-instance YouTube URLs, and the reconstruction audit records a download success rate of 65.9 % (16,816 of 25,513 clips). This constrained the effective number of usable training examples and determined the feasible subset definitions, including the use of MS-ASL50 instead of the more standard MS-ASL30 baseline used in WLASL-style comparisons, since only six MS-ASL classes met the train/val/test sample threshold needed for an MS-ASL30 reproduction (cf. Section 6.6).
3. **Limited development timeline:** The executed work plan ran from 5 January 2026 to the final submission window in May 2026. This fixed calendar limited how many repeated experiments could be completed. Multi-seed evaluation was applied to the main reported configurations, but broader vocabulary-scaling runs and additional cross-dataset architecture comparisons were not included in the experimental scope.
4. **Linguistic scope:** The project validates a single sign language, American Sign Language, and uses ASL exclusively for both the WLASL and MS-ASL benchmarks. The pipeline contains no ASL-specific rule-based component, and each component that consumes landmarks could in principle be retrained on a different sign language given a comparable dataset. That validation has not been performed, so transfer to Spanish Sign Language, British Sign Language or any other sign language remains future work.

Chapter 5

Technical Development

This chapter describes the research and engineering foundations of this FDP: (i) how the recognition solution is structured into independent stages, the methodological investigations that informed the choice of algorithms and hyperparameters, (ii) and the functional capabilities provided by each stage. It also details the execution environment and the requirements that drove the architectural design. Finally, it describes the experimental datasets that feed the recognition pipeline.

5.1 Solution design

The solution is structured as a modular three-stage architecture, integrating research-driven methods across data preparation, model training and evaluation, and runtime inference. The stages are physically separated (different processes and different machines in the execution environment) but share a single source of truth for data formats and trained model artefacts, so a checkpoint produced during model training can be served unchanged during runtime inference. Figure 5.1 shows the solution architecture and its execution flow.

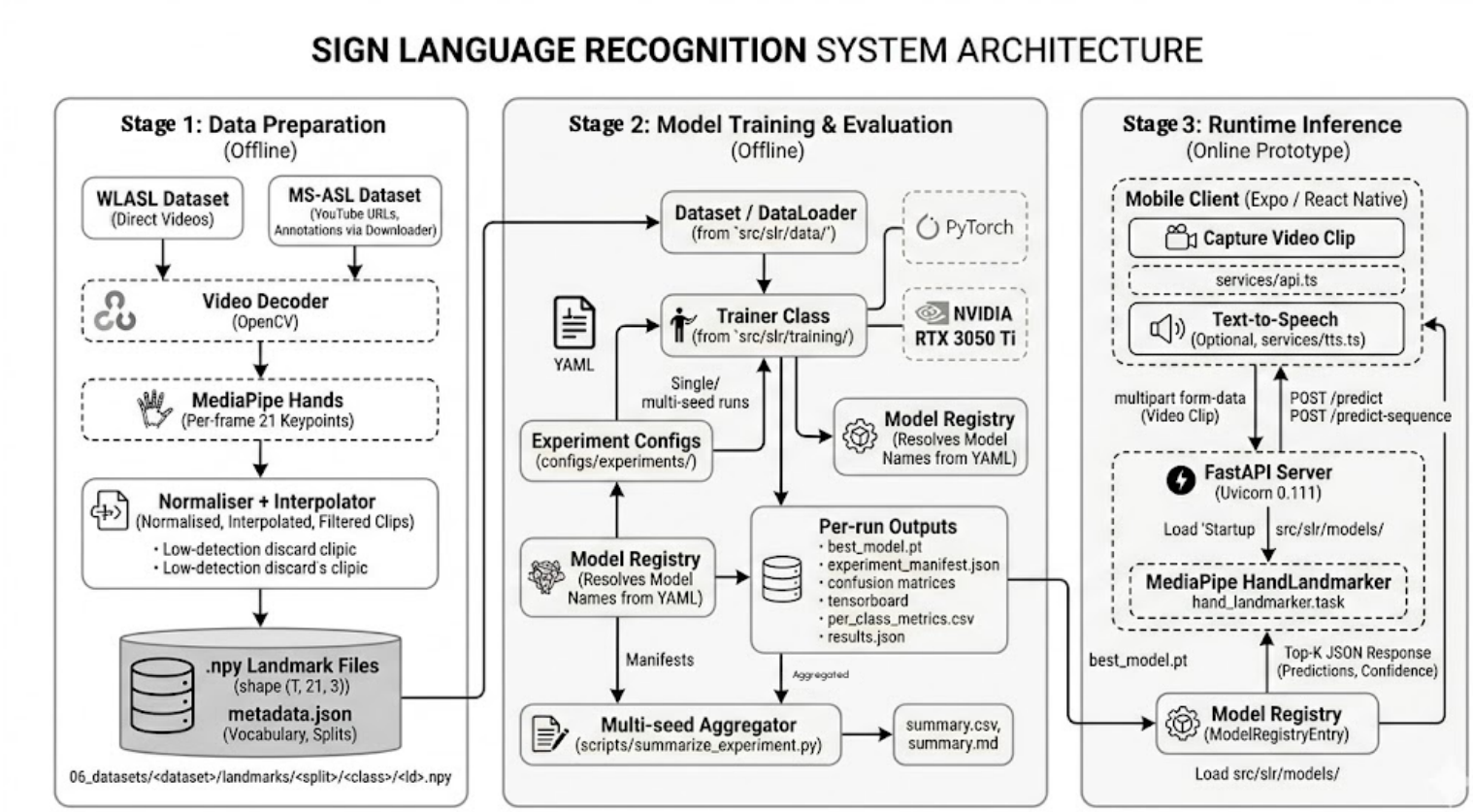


Figure 5.1: Solution architecture showing the data preparation, training and evaluation, and runtime inference stages, based on the implemented pipeline.[1]

[1] Image generated by Google Gemini, Google LLC, 2026, from a prompt supplied that described the three-stage Sign Language Recognition system architecture and requesting a professional, minimalist diagram for a research paper.

5.1.1 Stage 1: Data preparation

The data preparation stage converts raw benchmark videos into landmark sequences that can be used by the training and inference stages. It corresponds to the first two stages of the recognition pipeline introduced in Section 4.2.2, namely video acquisition and decoding, and landmark extraction and normalisation. This section decomposes those stages into five execution phases:

1. **Video acquisition:** The input is the official Word-Level American Sign Language (WLASL) distribution or the Microsoft American Sign Language (MS-ASL) annotation files. WLASL clips are downloaded from the dataset distribution, while MS-ASL clips are reconstructed from the referenced YouTube videos. The output is a local set of labelled video clips aligned with their gloss and split metadata.
2. **Clip decoding and temporal trimming:** OpenCV decodes each video frame by frame. For MS-ASL, each source video is trimmed with the per-instance `(start_time, end_time)` pair. The output is an ordered frame sequence for one sign instance.
3. **Hand landmark extraction:** MediaPipe Hands estimates 21 three-dimensional landmarks per valid frame. The output for each frame is a structured hand representation with coordinates for the wrist, finger joints and fingertips.
4. **Geometric normalisation and missing-frame handling:** Each frame is wrist-centred and length-normalised by the wrist-to-middle-finger distance. Frames where MediaPipe fails to detect a hand are linearly interpolated from neighbouring valid frames, and clips whose detection rate falls below 0.3 are discarded.
5. **Serialisation:** Each surviving clip is stored as a per-clip `*.npy` array of shape $(T, 21, 3)$. The accompanying `metadata.json` records the dataset name, gloss vocabulary, split assignments and source-video identifiers.

This stage is implemented in `src/slr/data/` and exposed through the `scripts/extract_landmarks_*.py` entry points.

5.1.2 Stage 2: Model training and evaluation

The training and evaluation stage turns landmark sequences into reproducible model comparisons. Its execution follows four phases:

1. **Configuration resolution:** The input is a YAML configuration under `configs/` folder. The file specifies the dataset path, model identifier, optimiser, sequence length and evaluation settings. Shared defaults are declared in `configs/base/gru_attention.yaml`, and experiment-specific files override only the fields that change.
2. **Model construction:** The registry in `src/slr/models/` resolves the model identifier into one of the implemented sequence architectures. The supported identifiers are `gru_baseline`, `gru_attention` and `transformer`.
3. **Optimisation and evaluation:** The trainer in `training/trainer.py` optimises the selected architecture and evaluates it through the shared evaluator in `evaluation/`. The same trainer supports the multi-seed protocol used to report mean and standard deviation in the experiments chapter.

4. **Artefact generation:** Each run produces a self-contained directory with the best checkpoint by validation Top-1, the training history, confusion matrices, per-class metrics and an `experiment_manifest.json` recording git commit, hardware, runtime, hyperparameters and final metrics.

The experiment-tracking code is implemented in `tracking/manifest.py` and `tracking/git_utils.py`.

5.1.3 Stage 3: Runtime inference

The runtime inference stage integrates the trained recogniser into a mobile/server proof of concept. Its execution follows five phases:

1. **Video capture:** The React Native and Expo client in `03_app/signtotext-expo/` records a short video clip with the device camera.
2. **Request submission:** The client sends the clip to the FastAPI server in `04_server/main.py` through the REST client implemented in `services/api.ts`.
3. **Server-side preprocessing:** The server applies the same MediaPipe landmark extraction, normalisation and sequence preparation used in the data preparation stage.
4. **Prediction:** The server loads the requested checkpoint from the model registry and returns Top-K predictions with confidence scores through `POST /predict`. The `POST /predict-sequence` endpoint provides a heuristic continuous-sequence variant.
5. **Client response:** The client displays the prediction shortlist and can pass the Top-1 label to the text-to-speech service in `services/tts.ts`.

The server reuses the `src/slr/` package directly, so the inference code path remains aligned with the evaluation code path.

5.1.4 Description of functional features

The solution exposes six high-level capabilities. Each one is grounded in a specific technical decision, thereby serving as a comprehensive record of the architectural evolution of the system.

- **F1 - Landmark-based input pipeline:** Each frame is reduced from a 224×224 RGB tensor to 63 floating-point landmark coordinates, a roughly 2,389$\times$ dimensionality reduction. This design choice discards background and skin-tone signal at source, which both shrinks the model and disconnects recognition accuracy from appearance bias.
- **F2 - Multi-seed training protocol:** The trainer supports running the same configuration across multiple random seeds and aggregating the metrics with mean and standard deviation. Without this protocol, a single-seed Top-1 figure on a small test set is indistinguishable from noise; with it, a 2 to 3 percentage-point standard deviation can be reported alongside every headline number.
- **F3 - Architecture-agnostic training loop:** The same `Trainer` accepts three model identifiers through a registry lookup: `gru_baseline` is a bidirectional GRU architecture without attention, `gru_attention` is the selected GRU architecture with a temporal attention layer, and `transformer` is a self-attention architecture used as a comparison baseline. This makes

the architecture comparison reproducible: only the model field of the YAML changes between runs, every other variable stays fixed.

- **F4 - Cross-dataset validation:** The same pipeline runs unmodified on WLASL and on MS-ASL. Only the dataset path and the gloss vocabulary differ between the two configurations; nothing in the model, optimiser or loss is dataset-specific, which is what makes the cross-dataset comparison fair.
- **F5 - Reproducible experiment tracking:** Every run writes an `experiment_manifest.json` that captures git commit, dataset metadata, hyperparameters, hardware, runtime, and final metrics in one place. A future user can reconstruct the exact configuration of any historical run from this single file.
- **F6 - Top-K predictions in inference:** The runtime API returns Top-1 and Top-5 candidates with confidence scores instead of forcing a single answer. This matches the way the model itself is evaluated and lets the client surface a shortlist when the Top-1 confidence is low.

For example, the `experiment_manifest.json` generated for the MS-ASL50 seed 2026 run records the dataset identifier, random seed, split sizes, GRU+Attention configuration, hardware, runtime, checkpoint path and final metrics, including 71.19% Top-1 and 91.53% Top-5 accuracy. This is the concrete output produced by the reproducible experiment-tracking feature.

5.1.5 Operational Environment

Training and landmark extraction were performed in the local development environment; the runtime prototype was tested with a paired mobile client and an HTTP server. Table 5.1 summarises the hardware, software runtime and storage formats under which the results in this FDP were produced.

Table 5.1: Operational environment for training, landmark extraction, and runtime inference.

Environment area	**Specification**
Operating system	Ubuntu 24.04 LTS (training and server); cross-platform (mobile client).
CPU	Multi-core x86_64 (local laptop).
GPU	NVIDIA RTX 3050 Ti Laptop, 4 GB VRAM, CUDA 12.x.
Python	3.10 (project); 3.11 supported via `requires-python = ">=3.10"`.
Deep learning framework	PyTorch 2.x with CUDA support.
Pose estimation	MediaPipe Hands 0.10+ (`HandLandmarker`).
Video processing	OpenCV 4.8+.
Mobile client framework	React Native + Expo.
Backend	FastAPI 0.111 with Uvicorn 0.29.
Persistence	JSON manifests, `.npy` landmark files, `.pt` PyTorch checkpoints.
Version control	Git.

The detailed installation and execution procedure is not repeated in this section. It is provided in Appendix B, where the environment setup, dataset preparation, training commands, evaluation commands and server execution procedure are described together.

5.2 Requirements

5.2.1 Description of requirement attributes

For the textual description of the different identified requirements, a set of attributes has been selected to describe each requirement. Below is a description of the meaning of each attribute used:

- **Code**: A unique abbreviated identifier for the requirement. It is constructed using the requirement type code followed by a hyphen and three digits. Requirements are divided into functional and non-functional requirements. The codes used are **FR** for Functional Requirements and **NFR** for Non-Functional Requirements. For example: `FR-001`.
- **Name**: Extended identification of the requirement.
- **Description**: A basic description of the identified requirement.
- **Source**: Indicates the source through which the requirement was identified. This value usually corresponds to one or more use case codes.
- **Necessity**: Determines the degree of implementation required for the requirement. The possible values are:
  - **Essential**: The requirement must be implemented.
  - **Desirable**: It is preferable to implement the requirement, but it is not mandatory.
  - **Optional**: The requirement may be implemented, but it is neither important nor mandatory.
- **Priority**: Defines the importance of the requirement, allowing the order in which requirements are included in the design and implementation process to be established. The possible values are:
  - **High**: The requirement must be implemented in the early stages of development.
  - **Medium**: The requirement should be implemented once high-priority requirements have been completed.
  - **Low**: The requirement should be implemented in the final stages of development. These requirements do not affect the correct functioning of the system.
- **Stability**: Defines the stability of the requirement throughout the software life cycle. This indicates whether the requirement may be modified during the life cycle. The possible values are:
  - **Stable**: The requirement will not change during the system life cycle.
  - **Unstable**: The requirement may change during the system life cycle.
- **Verifiability**: Defines the degree to which a requirement can be verified, that is, the extent to which it is possible to confirm that the requirement has been implemented in the developed system. The possible values are:
  - **High**: It is possible to verify that the requirement has been implemented. These requirements usually correspond to core system functionalities.

- **Medium**: It is possible to verify that the requirement has been implemented, but it requires complex testing or inspection of the source code.
- **Low**: It is difficult, or in some cases impossible, to verify whether the requirement has been implemented.

5.2.2 Functional Requirements

The system specifies four functional requirements covering the core recognition pipeline (FR-001, FR-002), the experiment-tracking infrastructure (FR-003), and statistical validation through multi-seed training (FR-004).

System requirement			
ID	FR-001	**Source**	Pipeline design
Name	Landmark extraction from video clips		
Description	The system shall extract 21 three-dimensional hand landmarks per frame from each input video clip using MediaPipe Hands, producing a per-clip array of shape $(T, 21, 3)$ stored as a `.npy` file alongside dataset metadata.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.2: FR-001: Landmark extraction from video clips

System requirement			
ID	FR-002	**Source**	Project objective
Name	Word-level sign classification		
Description	The system shall classify input video clips into one of the N predefined ASL word classes for the active dataset, returning Top-1 and Top-5 predictions accompanied by confidence scores.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.3: FR-002: Word-level sign classification

System requirement			
ID	FR-003	**Source**	Reproducibility requirements
Name	Reproducible experiment tracking		
Description	Every training run shall generate a structured manifest capturing dataset version, hyperparameters, git commit, hardware, runtime, and final metrics in a single JSON file, sufficient to reconstruct the run configuration without consulting external sources.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.4: FR-003: Reproducible experiment tracking

System requirement			
ID	FR-004	**Source**	Statistical validation
Name	Multi-seed training support		
Description	The training subsystem shall support running the same configuration across multiple random seeds and aggregating the resulting metrics with mean and standard deviation, so that every reported result can be accompanied by an uncertainty estimate.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	High

Table 5.5: FR-004: Multi-seed training support

5.2.3 Non-functional Requirements

Three non-functional requirements characterise quality attributes of the system: runtime performance (NFR-001), code quality and modularity (NFR-002), and documentation completeness (NFR-003).

System requirement			
ID	NFR-001	**Source**	User experience
Name	Inference latency		
Description	End-to-end inference for a single video clip shall complete in under two seconds on the target deployment hardware (mobile client recording the clip, server running landmark extraction and model prediction).		
Necessity	Desirable	**Priority**	Medium
Stability	Unstable	**Verifiability**	Medium

Table 5.6: NFR-001: Inference latency

System requirement			
ID	NFR-002	**Source**	Maintainability
Name	Modular code architecture		
Description	The codebase shall be organised as a Python package with separate submodules for data, models, training, evaluation, tracking, and utilities, supporting independent unit testing of each submodule.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.7: NFR-002: Modular code architecture

System requirement			
ID	NFR-003	**Source**	Academic rigor
Name	Documentation completeness		
Description	The repository shall include a reproducibility guide that enables a third party to replicate the headline experimental results with documented commands and dependency declarations.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	Medium

Table 5.8: NFR-003: Documentation completeness

5.3 Datasets

This FDP spans two recognition tasks with different data requirements: hand localisation from still images and word-level sign language recognition from video. Six datasets are therefore used across the experimental pipeline. This section formalises the properties of each dataset by first defining a common set of descriptive attributes, then presenting a comparative table and finally describing each dataset with the same structure: metadata and technical specifications, composition and volume, source, and preparation.

5.3.1 Dataset Attribute Definitions

To ensure a consistent and unambiguous characterisation of every dataset, the following attributes are used in Table 5.9. Each attribute captures a distinct dimension relevant to dataset selection, reproducibility, and the interpretation of experimental results:

- **ID**: A unique alphanumeric identifier assigned to each dataset within this FDP. The prefix `DS-` followed by a two-digit index allows unambiguous cross-referencing in subsequent chapters when reporting experimental configurations and results.
- **Name**: The official or widely-used name of the dataset as it appears in the literature or on its hosting platform.
- **Year**: The year of original publication or first public release. This contextualises the dataset within the chronological development of the field and helps evaluate whether it reflects current data collection practice.
- **Task**: The specific sub-task for which the dataset was designed. Three task categories appear in this FDP:
  - **Hand Detection**: the dataset provides annotated bounding boxes or segmentation masks marking the location of hands in images.
  - **Letter Recognition**: the dataset provides labelled static images of individual American Sign Language (ASL) letter signs.
  - **Word Recognition**: the dataset provides labelled video clips of complete ASL word signs, requiring temporal modelling.
- **Samples**: The total number of annotated data instances. For image datasets, this equals the number of images; for video datasets, it equals the number of individual video clips. Where a

subset of the full dataset was used in this FDP, this is noted in the individual descriptions.

- **Classes**: The number of distinct output categories. For hand detection datasets this is always 1 (the single *hand* class); for sign language datasets it corresponds to vocabulary size, meaning the number of distinct letters or word glosses.
- **Format**: The storage format and annotation type of the raw data. This attribute specifies whether instances are static images or video clips, the image resolution where relevant, and the form of ground-truth annotation, such as bounding box coordinates in XML, pixel-level segmentation masks, image class labels or video-level class labels.
- **Input Representation**: Whether the models trained on this dataset consume *pixel-based* data (raw RGB or greyscale image/video frames) or *landmark-based* data (structured three-dimensional keypoint coordinates extracted by a pose estimator such as MediaPipe). Pixel-based inputs preserve full visual detail but are high-dimensional and sensitive to background, lighting and signer appearance. Landmark-based inputs are compact and less exposed to visual variability, at the cost of depending on a reliable keypoint extractor.

5.3.2 Comparative Overview

Table 5.9 provides a high-level comparison of the six datasets used in this FDP. Datasets DS-01 and DS-02 support the hand localisation pipeline. Datasets DS-03 and DS-04 were used in the initial letter-level classification phase, which was later superseded by word-level recognition. Datasets DS-05 and DS-06 are the video benchmarks used for the final landmark-based ASL recognition pipeline.

Table 5.9: Comparative overview of all datasets used in this FDP.

ID	Name	Year	Task	Samples	Classes	Format
DS-01	EgoHands	2015	Hand detection	4,800	1	Images + segmentation masks
DS-02	Kaggle Hand Dataset	2023	Hand detection	∼4,200	1	Images + bounding boxes (XML)
DS-03	Sign Language MNIST	2017	Letter recognition	34,627	24	28×28 greyscale CSV
DS-04	ASL Alphabet	2018	Letter recognition	87,000	29	200×200 RGB JPG
DS-05	WLASL	2020	Word recognition	21,083	2,000	RGB video clips
DS-06	MS-ASL	2018	Word recognition	25,513	1,000	RGB video clips, YouTube

5.3.3 Hand Detection Datasets

Datasets DS-01 and DS-02 are used jointly to train the hand localisation module based on YOLOv8n. Combining the two sources increases viewpoint diversity: an egocentric-only detector may fail when the camera is positioned at eye level, while a detector trained only on third-person images may struggle with first-person perspectives. After merging, duplicate removal and a stratified train/validation/test split, the combined training set comprises approximately 5,223 annotated images.

DS-01: EgoHands

1. **Metadata and technical specifications:** EgoHands, introduced by Bambach et al. [29], is an egocentric hand localisation dataset published in 2015. Ground-truth annotations are stored as pixel-level segmentation polygons in MATLAB `.mat` files. The input representation used in this FDP is pixel-based.
2. **Composition and volume:** The dataset contains approximately 4,800 labelled frames extracted from 48 video recordings. Most frames contain two visible hands and include overlap or occlusion cases during everyday interactions such as card games, chess and jigsaw puzzles.
3. **Source:** Indiana University collected the videos using a GoPro camera mounted on the chest of an observer watching another person. This first-person perspective makes the dataset useful for close-range hand detection but limits its viewpoint diversity.
4. **Preparation:** The segmentation polygons were converted to axis-aligned bounding boxes in YOLO format. EgoHands was then merged with DS-02 to reduce the first-person bias of the training corpus.

DS-02: Kaggle Hand Dataset

1. **Metadata and technical specifications:** The Kaggle Hand Dataset is a collection of annotated hand images published on Kaggle [30]. Ground-truth bounding boxes are provided in Pascal Visual Object Classes (VOC) XML format. The input representation used in this FDP is pixel-based.
2. **Composition and volume:** The dataset contains approximately 4,200 labelled images of hands in third-person perspectives. The images include varied backgrounds, lighting conditions, skin tones and partial occlusions.
3. **Source:** The dataset is distributed through the Kaggle platform and complements DS-01 by adding images that are not captured from the camera wearer's viewpoint.
4. **Preparation:** Bounding-box coordinates were normalised to the YOLO format used by the detection pipeline. The joint corpus formed by DS-01 and DS-02 yielded a YOLOv8n model with approximately 98% mean average precision at 50% overlap (mAP50) and 87% mean Intersection over Union (Mean IoU) on the held-out test split.

5.3.4 Letter Recognition Datasets

Datasets DS-03 and DS-04 were used during the initial phase of the FDP, when the recognition target was the static letter-level ASL alphabet. Both datasets cover 24 of the 26 ASL letters; the letters *J* and *Z* are excluded in both cases because they require hand motion and cannot be represented by a single static image. The two sources were merged and unified into a single training corpus for letter classification experiments. Although this phase was later superseded by the word-level recognition approach, these datasets remain relevant because they established the viability of data-driven letter classification and informed the design of the preprocessing pipeline. The input representation for both datasets is pixel-based.

DS-03: Sign Language MNIST

1. **Metadata and technical specifications:** Sign Language MNIST, hosted on Kaggle [31], mirrors the format of the MNIST handwritten digit benchmark. Each sample is a 28×28 greyscale image distributed in CSV format, with one label column and one pixel-intensity column per image position. The input representation is pixel-based.
2. **Composition and volume:** The dataset contains 27,455 training images and 7,172 test images, for a total of 34,627 samples. Each sample represents one of 24 static ASL letters.
3. **Source:** The dataset was constructed from ASL alphabet photographs that were rescaled and converted to the MNIST-style format.
4. **Preparation:** The low resolution and greyscale format made the dataset simple to load and normalise, but it also removed fine-grained finger articulation details. For the initial letter-recognition experiments, the dataset was restricted to the 24 classes shared with DS-04.

DS-04: ASL Alphabet

1. **Metadata and technical specifications:** ASL Alphabet, distributed on Kaggle [32], provides RGB images at a resolution of 200×200 pixels. The input representation is pixel-based.
2. **Composition and volume:** The dataset contains approximately 87,000 images organised into 29 classes: the 24 static ASL letters, the *space* class and two non-letter classes (*del*, *nothing*). Each class has 3,000 training images captured in a controlled environment.
3. **Source:** The images were collected for static ASL alphabet recognition and provide a higher-resolution complement to DS-03.
4. **Preparation:** For integration with DS-03, images were converted to greyscale and resized to a common format. The unified corpus contained approximately 79,000 training images and 7,000 test images covering the 24 shared letter classes.

5.3.5 Word Recognition Datasets

Datasets DS-05 and DS-06 are the primary benchmarks for word-level ASL recognition. WLASL is the dataset on which the architecture was tuned, while MS-ASL was added later to validate the same pipeline on an independent ASL corpus with a different signer distribution.

DS-05: WLASL

1. **Metadata and technical specifications:** The Word-Level American Sign Language (WLASL) dataset, published by Li et al. [1], contains 21,083 video instances covering 2,000 ASL glosses. Each instance is a short RGB video labelled with a single word-level gloss.
2. **Composition and volume:** Each clip shows a single signer performing one complete sign, typically against a plain or low-clutter background, with durations ranging from approximately one to four seconds. WLASL is partitioned into progressively larger evaluation subsets: WLASL100, WLASL300, WLASL1000 and WLASL2000.
3. **Source:** Videos were collected from multiple online platforms, including YouTube channels maintained by ASL educators, online courses and signing dictionaries. Qualified ASL experts

manually verified the sign production. A practical limitation is that WLASL relies on external video URLs, so some clips become unavailable over time.

4. **Preparation:** Raw RGB frames are not used as model input in this FDP. MediaPipe Hands [7] is applied to each frame to extract 21 three-dimensional hand landmarks, yielding 63 scalar features per frame. The effective detection rate of MediaPipe on WLASL clips is approximately 69%; frames where no hand is detected are linearly interpolated from adjacent detected frames, and each video is resampled to a fixed sequence length of 64 frames.

   For the experiments described in this FDP, the following WLASL subsets were extracted and used:

   - **WLASL30**: The 30 most frequent glosses, used for initial architecture exploration and model selection. Split: 242 training / 56 validation / 34 test videos.
   - **WLASL100**: The 100 most frequent glosses, used for K-Fold cross-validation and multi-split evaluation. Split: 741 training / 162 validation / 99 test videos.
   - **WLASL300, WLASL1000, WLASL2000**: Larger subsets targeted for progressive scaling experiments, where the goal is to assess model accuracy and generalisation at increasing vocabulary sizes.

DS-06: MS-ASL

1. **Metadata and technical specifications:** The Microsoft American Sign Language (MS-ASL) dataset, published by Vaezi Joze and Koller [2], defines 25,513 video instances over 1,000 glosses. The original release provides annotation JSON files rather than distributing the video clips directly.
2. **Composition and volume:** The official split contains 16,054 training samples, 5,287 validation samples and 4,172 test samples. The vocabulary is organised into four cumulative subsets (MS-ASL100, MS-ASL200, MS-ASL500 and MS-ASL1000) obtained by filtering on the integer label.
3. **Source:** Each annotation references a YouTube URL, a per-instance `(start_time, end_time)` interval and a normalised signer bounding box. Of the 25,513 declared instances, **16,816 were successfully downloaded (65.9% coverage)**, with split-level coverage of 74.0% / 47.5% / 58.0% for train, validation and test. The remaining clips were unavailable because the source videos had been removed or made private.
4. **Preparation:** A disk-level audit graded every downloaded clip by integrity and training viability. Only **6** of the top-30 classes met the strict MS-ASL30 threshold of 30 train / 5 validation / 5 test surviving samples, while **17** met the MS-ASL50 threshold of 20 / 5 / 5 and **57** met the MS-ASL100 threshold of 20 / 3 / 3. The chosen subset is therefore **MS-ASL50**, defined as the 50 classes with the largest training counts that satisfy the 20 / 3 / 3 threshold. The subset was processed with the same MediaPipe pipeline used for WLASL: each frame was reduced to 21 three-dimensional hand landmarks and stored as a per-clip `.npy` array. After extraction, the final per-split counts are **1,410 train / 306 validation / 236 test (1,952 total)**, with 31 source clips dropped because their mean detection rate fell below 0.3. The mean detection rate across the surviving clips is **83.9%**.

Chapter 6

Experiments

This chapter presents the experimental process chronologically, reflecting the project's evolution from its inception to final validation. The quantitative part is organised as seven phases. Each phase states what it tries to test, which configuration is used, which results are obtained, what the result contributes, what it does not solve, and whether the phase objective is achieved. This structure is intended to make the experimental reasoning understandable without requiring prior knowledge of the repository or the intermediate notebooks.

The chapter begins with the initial hand-detection and letter-classification pilot studies that informed the subsequent methodological pivot. The work then progressed into architectural benchmarking, sequence-length selection, WLASL30 validation, cross-dataset validation on MS-ASL50 and vocabulary scaling. The quantitative block closes with a balance of the evidence and its limitations.

Every result reported below is backed by stored experiment artefacts, including configurations, metrics, plots, trained checkpoints and an experiment manifest for each run. The detailed repository paths needed to reproduce or audit the results are centralised in the reproducibility guide and internal audit material, so the chapter cites experiment outputs conceptually rather than repeating long file paths in every caption.

6.1 Experimental Methodology

The experimental methodology is organised as a sequence of phases. Each phase has a defined purpose, configuration, measurement procedure, positive evidence, negative evidence and final balance. The comparative metrics used throughout the chapter are those defined in Section 4.2: Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy.

1. **M1 - Exploratory pilots:**
    - **Input:** EgoHands, Kaggle Hand Dataset, Sign Language MNIST and ASL Alphabet.
    - **Configuration and measurements:** YOLOv8n, SSDLite-MobileNetV3 and EfficientDet-D0 were compared for hand localisation, while CNN classifiers were trained for static ASL letters. Detection used mAP50, Mean IoU and latency; letter recognition used classification accuracy.

- **Output:** Evidence that detection and static letters were tractable but insufficient for word-level ASL recognition.

2. **M2 - Architecture selection:**
   - **Input:** WLASL30 landmark sequences, with 242 train, 56 validation and 34 test clips.
   - **Configuration and measurements:** Five locally tuned architecture families were compared under the same split, preprocessing and hardware. Each run was evaluated on the fixed test split.
   - **Output:** Selection of the best-performing GRU+Attention configuration for downstream experiments.
3. **M3 - Sequence-length sweep:**
   - **Input:** WLASL30 landmark sequences and the selected GRU+Attention architecture.
   - **Configuration and measurements:** Three maximum sequence lengths were tested, 32, 64 and 96 frames, each with seeds 42, 123 and 2026.
   - **Output:** Selection of `max_frames`=64 as the default temporal input length.
4. **M4 - WLASL30 validation:**
   - **Input:** The WLASL30 fixed split and WLASL100 filtered to the same 30 glosses.
   - **Configuration and measurements:** The fixed split used three seeds, while the complementary validation used Stratified 5-Fold Cross-Validation with class distribution preserved.
   - **Output:** Mean and standard deviation estimates for the selected configuration under seed and split variation.
5. **M5 - Cross-dataset validation:**
   - **Input:** MS-ASL50 landmark sequences, with 1,410 train, 306 validation and 236 test clips.
   - **Configuration and measurements:** The selected configuration was retrained with the same preprocessing and three seeds, changing only the dataset path and vocabulary.
   - **Output:** Evidence about whether the pipeline reproduced on a second ASL corpus.
6. **M6 - Vocabulary scaling:**
   - **Input:** WLASL subsets from 100 to 1,500 classes, plus the 30-class reference.
   - **Configuration and measurements:** The selected configuration was retrained with `max_frames`=64 and seed 42. Larger subsets used the extended budgets recorded in their manifests.
   - **Output:** A scaling curve relating vocabulary size, per-class data availability and test performance.
7. **M7 - Quantitative balance and limitations:**
   - **Input:** The results and artefacts produced by phases M1 to M6.

- **Configuration and measurements:** No new model is trained. The phase classifies the limitations observed during the previous experiments and separates supported conclusions from claims that remain outside the evidence.
- **Output:** A final assessment of what the quantitative experiments prove, what they do not prove and which objectives are only partially satisfied.

Unless a phase states otherwise, training and evaluation execute on the local workstation described in Section 5.1.5. The standard optimisation schedule uses AdamW with learning rate 1×10^{-3}, weight decay 5×10^{-4} and label smoothing of 0.1 on the cross-entropy loss. The learning rate follows a CosineAnnealingLR schedule. Early stopping monitors validation Top-1 with patience 30 epochs, and the maximum standard run length is 150 epochs.

Two reliability protocols are used when the sample size permits it. The **multi-seed protocol** repeats the same configuration with seeds 42, 123 and 2026, then reports mean $\pm$ standard deviation. This is required on WLASL30 because the test split contains only 34 samples, so one error changes Top-1 by approximately 2.94 percentage points. The **Stratified 5-Fold Cross-Validation** protocol uses WLASL100 filtered to the same 30 classes, 332 total samples and five non-overlapping folds with class distribution preserved. It complements the multi-seed protocol by varying the test split itself rather than only the model initialisation.

Test-Time Augmentation (TTA) was excluded from the evaluation protocol. Earlier exploratory runs on WLASL30 showed that TTA increased the reported Top-1 without a corresponding gain in true generalisation. Every number in this chapter therefore reflects a single forward pass per test sample.

Part I: Quantitative Evaluation

6.2 Phase 1 - Exploratory Work and Methodological Pivot

Purpose: The purpose of the first phase is not to obtain the final recognition result. Its role is to decide whether the project should remain in a pixel-based hand-detection and letter-classification path, or pivot towards word-level recognition from hand landmarks. The phase therefore evaluates two simpler problems first: locating hands in images and recognising static ASL letters.

Configuration: Table 6.1 summarises the two exploratory branches. The table is intentionally separated from the later result tables because these pilots do not use the same benchmark, task or metric as the final word-recognition experiments.

Table 6.1: Configuration of the exploratory pilots. Source: self-created from the dataset chapter and stored exploratory artefacts.

Branch	**Input data**	**Configuration**	**Measurement and decision criterion**
Hand localisation	EgoHands + Kaggle Hand Dataset, approximately 5,223 annotated images after merging and splitting.	YOLOv8n compared with SSDLite-MobileNetV3 and EfficientDet-D0.	Detection quality through mAP@0.5 and Mean IoU, plus practical latency. The branch passes if hand localisation is reliable enough not to be the main bottleneck.
Static letter recognition	Sign Language MNIST + ASL Alphabet, restricted to static ASL letters.	CNN-based image classifiers over static hand-shape images.	Classification accuracy on isolated letters. The branch passes only as a feasibility pilot, because letters *J* and *Z* require motion and are outside a static-frame formulation.

Results: The hand-localisation experiment produced a YOLOv8n detector with approximately **98 % mAP@0.5** and **87 % Mean IoU** on the held-out test split. This confirmed that locating hands was technically feasible. In parallel, the static-letter classifiers also achieved high accuracy on the simplified alphabet task, but the task itself was too far from the final objective: recognising a word from a temporal ASL clip.

Positive evidence: The positive contribution of this phase is that it de-risks the front of the pipeline. It shows that the project does not fail because hands cannot be detected or because hand-shape information is impossible to learn. It also provides evidence that a visual pipeline can extract useful signal from ASL-related images.

Negative evidence: The limitation is that neither pilot solves word-level sign recognition. Static images cannot represent signs whose meaning depends on movement, and an RGB sequence model would have to learn to ignore background, lighting and signer appearance from a very small number of clips per class. The pilot therefore exposes a representation problem rather than a model-selection problem.

Conclusion: The phase achieves its objective as an exploratory phase. It does not produce a deployable recogniser, but it justifies the methodological pivot used in the rest of the chapter: each frame is represented by 21 three-dimensional MediaPipe landmarks, flattened into 63 features, instead of by a 224×224 RGB image. This reduces the input dimensionality by approximately $2{,}389\times$ and moves the learning problem from appearance-based recognition to geometric sequence classification.

6.3 Phase 2 - Architecture Selection on WLASL30

Purpose: The purpose of this phase is to choose the architecture family that best converts one-hand MediaPipe landmark sequences into word-level ASL predictions. The question is specific: among the locally implemented and tuned candidates, which architecture should be carried forward to the validation phases?

Configuration: Table 6.2 defines the fixed comparison setup. The architecture itself is the variable under test; the dataset, preprocessing and training policy remain fixed except for the Transformer optimiser settings documented in the table note.

Table 6.2: Configuration used for architecture selection. Source: self-created from experiment manifests and configuration files.

Element	Value
Dataset	WLASL30, the 30 most frequent WLASL classes: 242 train, 56 validation and 34 test clips.
Input representation	One detected hand per frame, represented by 21 MediaPipe landmarks with 3 coordinates each, flattened to 63 features.
Temporal length	64 frames per clip, using uniform sampling or padding as required.
Compared architectures	LSTM, GRU, GRU with enriched geometric features, GRU+Attention and Transformer.
Shared training policy	AdamW, cross-entropy with label smoothing, cosine learning-rate schedule, batch size 32, early stopping on validation Top-1.
Selection criterion	Best fixed-split test performance, with Top-1 accuracy as the primary metric and Top-5 accuracy as a secondary usability-oriented metric.

Table 6.3: Single-split test results of the five locally tuned architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the selected configuration is in Section 6.5.

Architecture	Top-1 (%)	Top-5 (%)	Notes
LSTM (landmarks)	41.18	67.65	Bidirectional, hidden size 256, 2 recurrent layers.
GRU (landmarks)	38.24	76.47	Bidirectional, hidden size 256, 2 recurrent layers.
GRU + Enriched features	47.06	79.41	GRU baseline with bone vectors and joint angles.
GRU + Attention	**58.82**	**91.18**	Selected configuration, 2.26 M parameters.
Transformer[a]	50.00	73.53	Four encoder layers, 8 heads, $d_{\text{model}} = 256$.

[a] *The Transformer baseline is the only exception in optimiser scale, using* $d_{\text{model}} = 256$, $\text{nhead} = 8$, *four transformer layers,* $\text{lr} = 5 \times 10^{-4}$, $\text{wd} = 1 \times 10^{-4}$ *and dropout* 0.3, *as recorded in its experiment manifest.*

Results: Table 6.3 shows that GRU+Attention is the best candidate in this controlled comparison: 58.82 % Top-1 and 91.18 % Top-5. The attention layer improves over the plain GRU by 20.58 percentage points in Top-1 and over the enriched-feature GRU by 11.76 percentage points. The Transformer baseline reaches 50.00 % Top-1, which is competitive but below the selected recurrent model on both Top-1 and Top-5.

Positive evidence: The phase supports the choice of a recurrent model with temporal attention. This is coherent with the structure of isolated sign clips: only some frames contain the most discriminative hand configuration or motion phase, and the attention head can weight those frames more heavily than preparation or recovery frames.

Negative evidence: The comparison is still a single-split result on only 34 test samples, so one error changes Top-1 by approximately 2.94 percentage points. In addition, the table cannot report Macro-F1 or balanced accuracy for all non-selected candidates because the stored comparison artefact only contains Top-1 and Top-5 for those rows. The result should therefore be read as architecture selection, not as a final statistical validation.

Conclusion: The phase achieves its objective. The selected architecture is a bidirectional GRU with hidden size 256, two layers, dropout 0.4 and a Temporal Attention head. This configuration becomes the default for every downstream quantitative experiment.

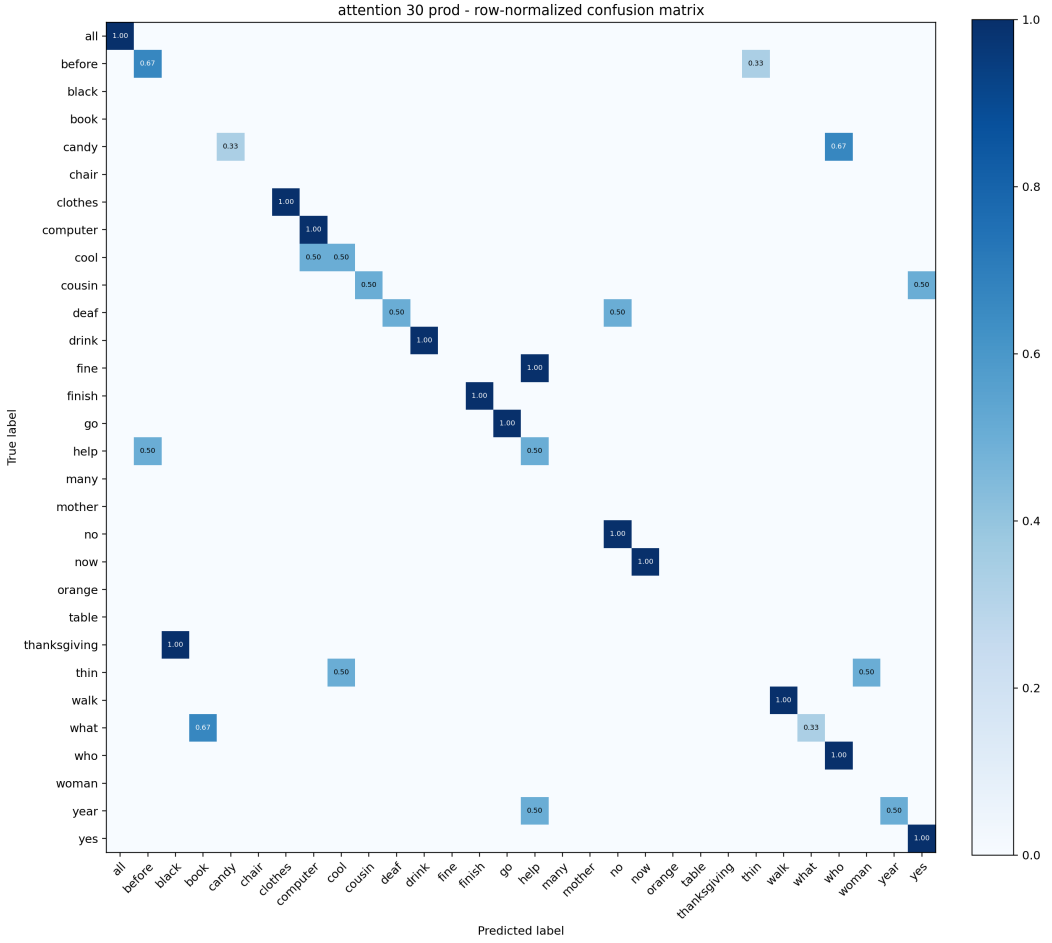


Figure 6.1: Row-normalised confusion matrix of the selected GRU+Attention configuration on the WLASL30 test split (34 samples). Source: self-created from experiment outputs.

Confusion-matrix reading: Figure 6.1 is used to inspect the selected model rather than to change the architecture decision. The matrix shows that the strongest errors are concentrated in low-support rows and in visually similar signs. Table 6.4 extracts the most relevant examples from the stored confusion artefacts.

Table 6.4: Most relevant confusion examples observed in the selected GRU+Attention WLASL30 run. Source: self-created from stored experiment outputs.

Ground truth	Predicted as	Errors	Interpretation
candy	*who*	2 / 3	Repeated class-level confusion on a low-count test row.
what	*book*	2 / 3	Repeated class-level confusion on a low-count test row.
fine	*help*	1	Single-sample error; recall impact is high because the class has few test clips.
thanksgiving	*black*	1	Single-sample error; likely driven by overlapping hand pose or brief decisive frames.
cool	*computer*	1	Single-sample error; likely driven by overlapping hand pose or brief decisive frames.

6.4 Phase 3 - Sequence Length Sweep

Purpose: The purpose of this phase is to select the maximum temporal input length `max_frames`. This is the only hyperparameter in the reported pipeline that changes the shape of the input sequence. Too few frames may remove discriminative motion; too many frames may increase padding and noise in short clips.

Configuration: Table 6.5 gives the sweep design. The selected architecture from Phase 2 is fixed, and only the temporal length changes.

Table 6.5: Configuration of the sequence-length sweep. Source: self-created from sweep configuration files.

Element	Value
Dataset and model	WLASL30 with the selected GRU+Attention architecture.
Tested values	32, 64 and 96 frames.
Seeds	42, 123 and 2026 for each temporal length, giving nine runs.
Fixed settings	Same landmark representation, optimiser, loss, batch size and early-stopping rule as Phase 2.
Selection criterion	Stability across seeds, prioritising Macro-F1 and balanced accuracy consistency alongside Top-1 accuracy.

Table 6.6: Sequence-length sweep on WLASL30 with 3 seeds per row. Mean ± standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity (≈2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs.

Frames	Top-1	Top-5	Macro-F1	Balanced Accuracy
32	56.86 ± 6.12	85.29 ± 2.95	50.52 ± 8.27	64.65 ± 7.48
64	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76
96	54.90 ± 4.49	81.37 ± 4.49	49.63 ± 6.33	61.36 ± 6.56

Results: The 64-frame configuration gives the highest Top-1 mean and the most stable per-class metrics. Its Macro-F1 standard deviation is 0.93 percentage points and its balanced accuracy standard deviation is 0.76 percentage points, both much lower than the corresponding values at 32 and 96 frames.

Positive evidence: The sweep supports 64 frames as a practical default. It preserves enough temporal information to outperform 32 and 96 frames on Top-1, while reducing seed-to-seed variation in Macro-F1 and balanced accuracy.

Negative evidence: The phase does not show that 64 frames is universally optimal. Both alternative settings have higher mean Top-5 accuracy, and the sweep is performed only on WLASL30. A dedicated sweep on MS-ASL50, where clips are longer on average, remains outside the executed experiments.

Conclusion: The phase achieves its objective for this project. The selected value is `max_frames`=64, which is fixed for the downstream WLASL30, MS-ASL50 and vocabulary-scaling experiments.

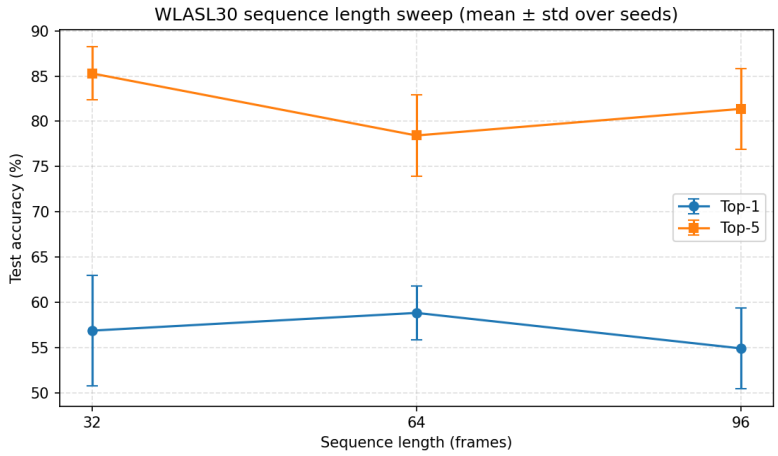


Figure 6.2: Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs.

6.5 Phase 4 - WLASL30 Validation

Purpose: The purpose of this phase is to validate the selected GRU+Attention pipeline on WLASL30 using protocols that reduce dependence on one random seed or one fixed test partition. This phase is the main answer to whether the selected configuration is reproducible on the dataset where it was tuned.

Configuration: Two complementary validation protocols are used. Table 6.7 shows why both are needed: one varies the split, and the other varies the model initialisation on the original fixed split.

Table 6.7: Validation protocols used for the selected GRU+Attention configuration on WLASL30. Source: self-created from k-fold and multi-seed outputs.

Protocol	Data used	What changes	Role in the phase
Stratified 5-Fold Cross-Validation	WLASL100 filtered to the same 30 classes, 332 total samples.	Five non-overlapping stratified test partitions.	Headline WLASL30 estimate because it smooths the effect of a single small test split.
Fixed-split multi-seed evaluation	WLASL30 split with 242 train, 56 validation and 34 test clips.	Seeds 42, 123 and 2026.	Measures seed sensitivity on the original benchmark split used by the architecture comparison.

Table 6.8: Aggregate WLASL30 validation results. K-fold artefacts store Top-1 and Top-5; Macro-F1 and balanced accuracy are therefore reported only for the fixed-split multi-seed protocol. Source: self-created from stored experiment outputs.

Protocol	Top-1	Top-5	Macro-F1	Balanced Accuracy
Stratified 5-Fold CV	63.85 ± 4.50	86.46 ± 2.04	N/A	N/A
Fixed split, 3 seeds	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76

Table 6.9: Per-seed test results of the selected GRU+Attention configuration on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Accuracy	Best Epoch
42	55.88	73.53	55.94	68.18	40
123	61.76	79.41	54.20	66.67	29
2026	58.82	82.35	54.49	67.42	27
Mean $\pm$ SD	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76	N/A

Results: The Stratified 5-Fold protocol reaches **63.85 % ± 4.50 %** Top-1 and **86.46 % ± 2.04 %** Top-5. The fixed-split multi-seed protocol reaches **58.82 % ± 2.94 %** Top-1 and **78.43 % ± 4.49 %** Top-5. The two numbers differ by approximately five percentage points because they answer different questions: the k-fold run varies the test partition, while the fixed-split run keeps the 34-sample test set and varies only the seed.

Positive evidence: The phase supports the reproducibility objective. The k-fold result is the strongest WLASL30 estimate because it averages over multiple stratified partitions, and the multi-seed result shows that the selected pipeline does not collapse under different initialisations.

Negative evidence: The fixed WLASL30 test split remains small. Classes such as *fine*, *thanksgiving* and *thin* have one or two supported test samples in the stored per-class metrics, so a single mistake can make their recall appear as zero. This limits the strength of per-class conclusions on WLASL30.

Conclusion: The phase achieves the WLASL30 validation objective, with an important qualification: the headline 63.85 % Top-1 result is a cross-validation estimate, while the 58.82 % Top-1 result is the stricter fixed-split multi-seed estimate. Both are reported because they describe complementary sources of uncertainty.

6.6 Phase 5 - Cross-Dataset Validation on MS-ASL50

Purpose: The purpose of this phase is to test whether the selected pipeline is tied to WLASL30 or can be retrained on an independent ASL dataset without changing the architecture. This phase therefore evaluates generalisation at the pipeline level, not zero-shot transfer: the model is trained again on MS-ASL50, but the preprocessing and architecture remain the same.

Configuration: Table 6.10 summarises the cross-dataset setup. The only intended changes are the dataset path and the output vocabulary.

Table 6.10: Configuration of the MS-ASL50 cross-dataset validation. Source: self-created from MS-ASL50 configuration files and experiment manifests.

Element	Value
Dataset	MS-ASL50, reconstructed from locally available MS-ASL clips.
Split after landmark extraction	1,410 train, 306 validation and 236 test clips.
Model and input	Same GRU+Attention model, 64 frames and 63 landmark features per frame as WLASL30.
Seeds	42, 123 and 2026.
Changed fields	Dataset location and gloss vocabulary. No architecture-specific or dataset-specific tuning is introduced.
Decision criterion	The phase passes if the same pipeline trains reproducibly on MS-ASL50 and reaches stable Top-1/Top-5 performance across seeds.

6.6.1 Building the MS-ASL50 subset

MS-ASL ships annotation JSONs and YouTube URLs rather than a fixed local video archive. The subset used in this project is therefore constrained by which clips were still downloadable when the local audit was executed. Table 6.11 records the relevant reconstruction facts.

Table 6.11: MS-ASL reconstruction and MS-ASL50 subset audit. Source: self-created from the MS-ASL download audit and subset extraction report.

Audit item	**Observed value**
Declared MS-ASL instances	25,513 clips in the original annotation files.
Successfully downloaded clips	16,816 clips, equivalent to 65.9 % of the declared instances.
Split-level download coverage	74.0 % train, 47.5 % validation and 58.0 % test.
Viability thresholds	6 top-30 classes meet 30 / 5 / 5; 17 classes meet 20 / 5 / 5; 57 classes meet 20 / 3 / 3.
Selected subset	50 classes with the largest train counts among classes satisfying the 20 / 3 / 3 threshold.
Landmark extraction output	31 clips dropped by the detection-rate floor, leaving 1,952 usable clips.
Mean MediaPipe detection rate	83.9 % across surviving MS-ASL50 clips.

The MS-ASL50 subset reproducible from the current YouTube state is therefore not identical to the subset that would have been reproducible when the dataset was released. Any direct comparison with published MS-ASL numbers must be contextualised by this time-dependent availability of source videos.

6.6.2 Cross-dataset results

Table 6.12: Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs.

Dataset	**Classes**	**Train / Val / Test**	**Top-1**	**Top-5**	**Macro-F1**
WLASL30	30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93
MS-ASL50	50	1,410 / 306 / 236	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44

Table 6.13: Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Accuracy	Best Epoch
42	66.95	89.83	64.22	66.34	55
123	72.46	90.25	70.81	72.75	120
2026	71.19	91.53	69.21	71.58	92
Mean ± SD	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44	70.22 ± 3.41	N/A

Results: The same GRU+Attention pipeline reaches **70.20 % ± 2.89 %** Top-1 and **90.54 % ± 0.89 %** Top-5 on MS-ASL50. The seed-to-seed Top-1 standard deviation is almost identical to WLASL30 (2.89 percentage points versus 2.94), even though the MS-ASL50 test set is substantially larger.

Positive evidence: The phase supports cross-dataset reproducibility. The model does not require a new architecture or dataset-specific tuning to train successfully on MS-ASL50. The larger test split also gives a finer measurement grid: one MS-ASL50 test error changes Top-1 by approximately 0.42 percentage points, compared with 2.94 percentage points on the fixed WLASL30 test split.

Negative evidence: The higher MS-ASL50 Top-1 value should not be interpreted as proof that MS-ASL is intrinsically easier. MS-ASL50 has more training clips, a larger test set and a higher MediaPipe detection rate than WLASL30. These structural differences are enough to explain part of the performance gap. The experiment proves reproduction of the pipeline, not a universal ranking of dataset difficulty.

Confusion behaviour: The full 50-class confusion matrix for the seed-2026 run is provided in Appendix A, Figure A.1. The stored top-confusion artefact reports the largest absolute confusions as *deaf* → *need* (4 of 14 samples, 28.6 %), *pencil* → *write* and *fish* → *lost* (each 3 of 5, 60.0 %), *nurse* → *sit* (3 of 6, 50.0 %), and *mother* → *woman* (2 of 4, 50.0 %). The per-class metrics file separately identifies three labels with zero recall on that run: *here*, *man* and *woman*. These errors indicate that some one-handed, location-similar signs still depend on fine-grained finger-shape cues that the one-hand landmark pipeline does not always separate.

Conclusion: The phase achieves its objective. It validates that the selected pipeline can be retrained on a second ASL corpus with stable performance. It does not remove the need for a larger, fully reproducible benchmark, because MS-ASL reconstruction depends on YouTube clip availability.

6.7 Phase 6 - Vocabulary Scaling

Purpose: The purpose of this phase is to measure how the selected pipeline behaves as the recognition vocabulary grows. The question is not whether the model can reach deployment-ready accuracy

at 1,500 classes; the question is where and how performance degrades when the number of classes increases and the mean number of videos per class decreases.

Configuration: Table 6.14 summarises the scaling setup. The 30-class row is the multi-seed WLASL30 reference from Phase 4. All larger-vocabulary rows are single-seed runs and must be interpreted as trend evidence rather than precise multi-seed estimates.

Table 6.14: Configuration of the WLASL vocabulary-scaling experiment. Source: self-created from scaling manifests and result files.

Element	**Value**
Model	Selected GRU+Attention pipeline with 64-frame landmark input.
Datasets	WLASL subsets with 30, 100, 300, 500, 700, 900, 1,000, 1,200 and 1,500 classes.
Seed policy	WLASL30 uses the three-seed reference; all larger subsets use seed 42.
Training budgets	Standard budget for smaller runs, with extended maximum budgets recorded in the manifests for larger subsets.
Primary output	Relationship between vocabulary size, available data per class and test Top-1 / Top-5 performance.
Decision criterion	The phase passes if it quantifies the scaling trend and identifies whether the bottleneck is vocabulary size, data scarcity or an implementation failure.

Table 6.15: Vocabulary scaling of the selected GRU+Attention configuration on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.8; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests.

Classes	**Train / Val / Test**	**Top-1**	**Top-5**	**Best Epoch**	**Train Time (s)**
30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	27–40	N/A
100	795 / 186 / 132	44.70	70.45	99	83
300	1,974 / 474 / 360	28.61	59.72	112	224
500	2,967 / 723 / 561	23.71	49.55	111	311
700	3,879 / 974 / 736	19.57	42.66	57	248
900	4,707 / 1,202 / 880	12.50	33.75	218	881
1,000	5,122 / 1,329 / 938	11.19	31.24	113	602
1,200	5,840 / 1,536 / 1,051	10.56	27.69	211	1,105
1,500	6,864 / 1,834 / 1,229	9.76	28.23	209	1,308

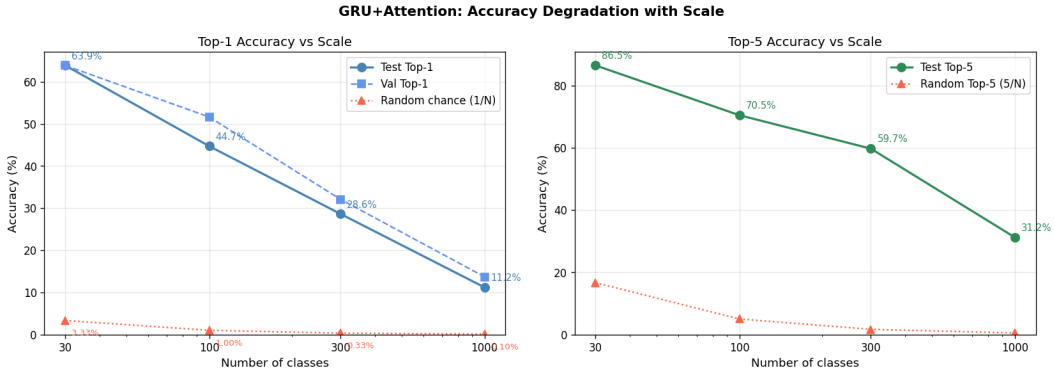


Figure 6.3: Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs.

Results: Top-1 accuracy decreases at every measured vocabulary size, from 58.82 % at 30 classes to 9.76 % at 1,500 classes. Top-5 also decreases strongly, although it rises slightly between 1,200 and 1,500 classes (27.69 to 28.23), which is another reminder that the large-vocabulary rows are single-seed measurements.

Positive evidence: The phase quantifies the scaling bottleneck instead of hiding it. The drop is gradual rather than an abrupt implementation failure, and it is consistent with the reduction in per-class data as the vocabulary expands. The parallel-coordinates diagnostic in Appendix A, Figure A.2, also supports this interpretation: the 30-class reference has 11.07 mean videos per class, 99.00 % final train accuracy and 63.90 % validation Top-1, while the 1,500-class run has 4.58 videos per class, 74.77 % final train accuracy and 11.67 % validation Top-1.

Negative evidence: The phase does not show that the selected model scales to large vocabularies in a practically usable way. Above 700 classes, several runs reach the maximum epoch budget without a clear improvement in validation performance. Because these rows are single-seed, the exact slope of the curve should not be treated as a statistically validated law.

Conclusion: The phase achieves its analytical objective but not a deployment objective. It shows that the main limitation is data starvation at the class level, not simply the number of labels or a single software failure. It also sets a clear boundary for the project: the implemented pipeline is appropriate as a research prototype and small-vocabulary recogniser, but it is not a large-vocabulary ASL translator.

6.8 Phase 7 - Quantitative Balance and Limitations

Purpose: The purpose of the final quantitative phase is to separate what the preceding experiments demonstrate from what remains unresolved. No new model is trained here. The phase summarises the positive evidence, the negative evidence and the practical meaning of the results with respect to the project objectives.

Configuration: The input to this phase is the set of artefacts generated by phases M1 to M6: configuration files, metrics, confusion matrices, per-class reports, manifests and audit notes. Table 6.16 classifies the limitations that affect interpretation.

Table 6.16: Limitation categories, ordered from data acquisition constraints to modelling choices. Source: self-created from the experimental audit.

Category	Where it appears	Effect on interpretation
Landmark extraction quality	WLASL and MS-ASL landmark preprocessing	Low MediaPipe detection rates reduce the number of informative frames available to the sequence model.
Evaluation resolution	WLASL30 fixed test split	Small test sets make single errors change Top-1 accuracy by several percentage points.
Dataset reconstruction	MS-ASL download audit	YouTube attrition means the reconstructed subset is time-dependent and not identical to the original release state.
Seed coverage	Vocabulary scaling beyond 30 classes	Single-seed runs show the scaling trend but not the exact seed variance.
Architecture coverage	Cross-dataset validation	Only the selected GRU+Attention configuration was retrained on MS-ASL50.
Training policy	Augmentation and class-imbalance handling	Fixed augmentation and unweighted loss leave minority-class effects unresolved.

Positive evidence: The quantitative experiments support four project claims. First, landmark-based input is a viable representation for isolated ASL word recognition. Second, temporal attention improves the recurrent baseline on WLASL30 under the executed comparison. Third, the same pipeline can be retrained on MS-ASL50 without dataset-specific architecture changes. Fourth, vocabulary growth exposes a data-scarcity bottleneck that can be measured directly.

Negative evidence: The experiments also constrain what can be claimed. The Transformer baseline was not dominant in the WLASL30 low-data setting, but this does not prove that Transformers are unsuitable for sign language recognition in general. The WLASL30 fixed test split is too small for precise per-class conclusions. The MS-ASL50 reconstruction must be interpreted in light of YouTube attrition. Finally, the scaling study beyond 30 classes is single-seed, so it establishes a downward trend but not a fully validated variance estimate.

Detailed limitations: The MediaPipe detection rate on WLASL is approximately 69 % at the frame level. Clips with motion blur, partially occluded hands or hands moving outside the frame produce silent failures that interpolation mitigates but does not erase. The same caveat applies less strongly to MS-ASL50, where the average detection rate is 83.9 %.

The WLASL30 test split contains 34 samples, so Top-1 accuracy has a resolution of approximately 2.94 percentage points per misclassification. The 5-Fold CV estimate partially mitigates this by averaging over five stratified partitions, but the fixed-split confusion matrix should not be read as a definitive per-class diagnosis.

YouTube attrition on MS-ASL is 34.1 % of declared source clips. The MS-ASL50 subset reproducible from the current YouTube state is therefore not identical to the one that was reproducible

when the dataset was released. Comparisons against the original MS-ASL release [2] must be contextualised by this difference in available clips.

Only the selected GRU+Attention configuration was evaluated cross-dataset. Running the specific Transformer baseline from Table 6.3 on MS-ASL50 would test whether its lower WLASL30 result reflects the low-data regime or the architecture choice itself. That experiment is a natural extension, but it was not executed within the scope of this project.

The augmentation pipeline was kept fixed throughout the chapter: rotation, scaling, translation, Gaussian noise and horizontal flip. Horizontal flip conflates left-handed and right-handed signers, which is acceptable for a small-vocabulary one-hand pipeline but would need re-examination for two-hand signs and for downstream studies of signer-handedness bias. Class-weighted loss and minority-class oversampling were also not applied, despite the per-class disparities documented in Sections 6.5 and 6.6.

Conclusion: The quantitative block passes the reproducibility objective for the selected pipeline: the model is selected through a controlled comparison, validated through repeated WLASL30 protocols and reproduced on MS-ASL50. The scalability objective is only partially achieved: the project measures the degradation from 30 to 1,500 WLASL classes, but it does not solve the large vocabulary setting. The final conclusion is therefore deliberately bounded: this work delivers a traceable, reproducible research prototype for isolated word-level ASL recognition from hand landmarks, not a deployment-ready general ASL translator.

Part II: Qualitative Evaluation

6.9 Qualitative Validation

The quantitative experiments in Part I evaluate recognition accuracy under controlled benchmark conditions. This section evaluates a different question: whether the SignToText prototype is understandable, responsive and useful when users interact with the application. The questionnaire therefore measures usability and perceived value, not the technical accuracy of the model, which was validated through the benchmark experiments above.

6.9.1 Survey Design

The qualitative validation used a short Google Form completed after participants tested the SignToText demo [33]. The form introduced the application as an ASL recognition prototype that translates detected signs into text and speech. It then collected three types of data: previous knowledge of sign language, Likert-scale usability ratings and open-ended feedback about confusion points or requested changes. The expected completion time was approximately five minutes.

The remaining survey questions were organised as follows:

1. **Likert-scale usability block:** This block used a 1 to 5 scale and evaluated five dimensions: first impression, understood as whether the participant quickly knew what the app did and how to start using it; clarity of the buttons and capture modes; comprehensibility of the prediction results; perceived response speed; and perceived utility.

2. **Feature-preference question:** This question allowed multiple selections, including automatic text-to-speech, Top-K alternatives with confidence percentages, temporal attention maps, model selection, continuous phrase mode, countdown before recording, gallery upload and processing statistics.
3. **Recommendation question:** This question used a 1 to 10 scale and was reduced to Net Promoter Score (NPS), computed as the percentage of promoters minus the percentage of detractors.

6.9.2 Results

The questionnaire collected responses from $N = 15$ participants. Table 6.17 summarises the Likert dimensions. The strongest dimension is perceived utility, with a mean of 4.5 out of 5, followed by perceived speed at 4.3 and first impression at 4.2. The lowest dimension is result comprehension, with a mean of 3.7, which indicates that the presentation of the prediction output still needs refinement.

Table 6.17: Usability questionnaire results for the SignToText demo. Means are computed on a 1 to 5 Likert scale. Source: self-created from Google Form responses.

Dimension	Mean score
First impression	4.2
Button and capture-mode clarity	4.0
Result comprehension	3.7
Perceived speed	4.3
Perceived utility	4.5

The recommendation item produced an NPS of $+40$. This is a positive usability signal for a research prototype, since more participants were willing to recommend the application than reject it. The result should not be interpreted as market validation because the sample is small and informal, but it does indicate that the interaction concept was received favourably.

Table 6.18 reports the three most valued features. Top-K alternatives with confidence percentages were selected by 87 % of participants, making them the most valued interface element. Automatic text-to-speech was selected by 73 %, and model selection by 60 %. These results support the design choice of exposing a prediction shortlist rather than presenting only the Top-1 label. Other evaluated features, including the temporal attention map, continuous phrase mode, countdown before recording, gallery upload and processing statistics, received markedly fewer selections.

Table 6.18: Most valued SignToText features according to the usability questionnaire. Participants could select multiple options. Source: self-created from Google Form responses.

Feature	Participants selecting it
Top-K alternatives with confidence percentages	87 %
Automatic text-to-speech	73 %
Selection between several trained models	60 %

6.9.3 Discussion

The qualitative results show that users understood the prototype's purpose and perceived it as useful. The high perceived-speed score also suggests that the end-to-end interaction did not feel slow during the demo.

The main friction appears in result comprehension. Free-text responses and observed user reactions showed that the temporal attention map and processing statistics confused several participants when shown without context. Prediction alternatives and speech output are useful to users, while diagnostic visualisations should be hidden behind an advanced-details control or accompanied by short explanatory labels.

Chapter 7

Ethical Implications and Social Impact

7.1 Ethical Impact of AI-Based Systems

An ethical analysis in AI examines the implications, concerns and challenges associated with the development and deployment of these systems. As AI is applied to increasingly sensitive contexts, technical evaluation must be paired with an explicit ethics review that assesses effects on users, data subjects and wider social contexts. This introductory review uses four technical dimensions commonly identified across AI ethics frameworks: **Human Rights and Fairness** (preventing bias and discrimination), **Accountability** (clarifying who is responsible for the system's decisions), **Privacy and Security** (protecting user data and confidentiality) and **Transparency** (avoiding opaque black-box behaviour). These dimensions are operationalised in Section 7.2 through the seven principles of Google's AI framework, which is the specific framework chosen for the project evaluation.

7.2 Ethical Evaluation Framework: Google's AI Principles

The framework **chosen** for this evaluation is **Google's AI Principles** [34], originally published in 2018 and refined annually; the primary reference for this analysis is the **2023 Progress Update** [35]. The framework was chosen because its seven principles are formulated as concrete objectives that map cleanly onto the design decisions of a research project. Google's annual progress reports demonstrate how the principles apply to systems that share characteristics with this work (computer vision and on-device inference). The seven principles are:

1. **Be socially beneficial:** address a real societal need and do so effectively.
2. **Avoid creating or reinforcing unfair bias:** ensure fair and inclusive treatment.
3. **Be built and tested for safety:** prioritise safety in design, validation and operation.
4. **Be accountable to people:** provide transparency and user control.
5. **Incorporate privacy design principles:** protect user data by design.

6. **Uphold high standards of scientific excellence:** follow rigorous methodology.
7. **Be made available for uses that accord with these principles:** prevent misuse and align deployment with the framework.

7.3 Project Analysis Against Google's AI Principles

This section evaluates the project against each principle using scored metrics. Each identifier uses the format Mx.y, where x denotes the principle number and y denotes the metric within that principle. Each metric uses a three-point scale: 0 (not addressed), 5 (partially addressed) and 10 (fully addressed). Averages are reported to one decimal place for consistency across principles. Some metrics derive from experimental results, whereas others reflect design decisions or documentation practices. Table 7.1 consolidates the scores, per-principle averages and global score.

7.3.1 Principle 1: Be Socially Beneficial

Sign language is the primary means of communication for millions of deaf and hard-of-hearing persons, yet significant communication barriers still exist in interactions with hearing people. An automatic American Sign Language (ASL) recognition system can act as a bridge in educational, medical and everyday contexts.

- **M1.1 - Real societal need (Score: 10):** The World Health Organization estimates over 430 million people worldwide have disabling hearing loss; the project targets a clear accessibility problem.
- **M1.2 - Effectiveness of the solution (Score: 5):** As summarised in Table 7.1, the best-performing configuration achieves **63.85% Top-1 / 86.46% Top-5 (5-fold cross-validation, WLASL30)**. The same configuration is reproduced under the modular pipeline as 58.82% $\pm$ 2.94% Top-1 across three seeds and extends to **70.20% $\pm$ 2.89% Top-1 / 90.54% $\pm$ 0.89% Top-5** on MS-ASL50. Accuracy degrades at scale (44.70% at 100 classes, 28.61% at 300, 11.19% at 1000), so the system remains a research prototype rather than a deployment-ready translator. This limitation supports an intermediate effectiveness assessment.
- **M1.3 - Accessibility of the technology (Score: 10):** Built on open-source tools (Python, PyTorch, MediaPipe) and public datasets (WLASL, MS-ASL), with the methodology documented in this work. The recognition pipeline was designed with mobile-first deployment in mind.

7.3.2 Principle 2: Avoid Creating or Reinforcing Unfair Bias

Sign language recognition is particularly bias-sensitive: signers vary in hand size, skin tone, signing speed and style. Bias arises both at the perception stage (hand detection and landmark extraction operating on raw images) and at the recognition stage.

- **M2.1 - Dataset diversity (word recognition) (Score: 5):** WLASL aggregates videos from multiple online sources, providing implicit signer variation, but lacks demographic annotations (skin tone, age, hand morphology), so diversity cannot be quantified formally.
- **M2.2 - Dataset diversity (hand detection and letter classification) (Score: 5):** The auxiliary datasets were:

1. EgoHands (∼4,800 egocentric images) for hand detection.
2. The Kaggle Hand Dataset (∼4,200 third-person images) for hand detection.
3. Sign Language MNIST (34,627 grayscale images for letter classification, with MNIST referring to the Modified National Institute of Standards and Technology dataset format).
4. The ASL Alphabet Dataset (∼87,000 RGB images) for letter classification.

None of these datasets provides demographic metadata, so potential biases cannot be ruled out.

- **M2.3 - Input representation fairness (Score: 10):** The word-recognition model consumes MediaPipe landmarks (21 keypoints $\times$ 3 coordinates) instead of raw pixels. Operating on geometric joint positions rather than visual appearance reduces sensitivity to skin colour, lighting and background, mitigating appearance-based bias at the recognition stage (though not at the upstream landmark extractor).
- **M2.4 - Per-class performance analysis (Score: 5):** Per-class metrics were logged across all experiments, surfacing wide accuracy gaps (e.g. "what" 8.3%, "cool" 12.5%, vs. "orange" 100%, "table" 100%). The disparities are mostly explained by visual similarity between signs and small per-class sample counts rather than by demographic factors. Class-weighted loss or oversampling were not applied in this iteration but are facilitated by the existing per-class logging.

7.3.3 Principle 3: Be Built and Tested for Safety

In this context, safety means producing reliable predictions and avoiding misleading outputs that could cause misunderstandings in sensitive domains.

- **M3.1 - Cross-validation methodology (Score: 10):** Evaluation uses Stratified 5-Fold Cross-Validation (mean Top-1 **63.85%** $\pm$ **4.50%**, mean Top-5 **86.46%** $\pm$ **2.04%**), corroborated by a 3-seed multi-seed sweep on the same dataset (58.82% $\pm$ 2.94% Top-1) under the refactored pipeline. Test-Time Augmentation was deliberately avoided because it inflates reported accuracy without reflecting true generalisation.
- **M3.2 - Scalability testing (Score: 10):** The model was retrained on progressively larger WLASL subsets (30 / 100 / 300 / 500 / 700 / 900 / 1000 / 1200 / 1500 classes), documenting accuracy degradation and an instability point above 700 classes. This stress-test establishes the boundaries within which the system can be considered reliable.
- **M3.3 - Hand detection reliability (Score: 10):** The YOLOv8n hand detector reached approximately 98% mAP50 on the combined test set, validated across both source datasets. A reliable detector is a prerequisite for the recognition pipeline.
- **M3.4 - Failure mode documentation (Score: 5):** Per-class confusion matrices identify systematically confused word pairs, but a confidence-threshold mechanism (so the system can abstain when uncertain) is not yet implemented and is needed before any production deployment.

7.3.4 Principle 4: Be Accountable to People

Accountability requires that the system is interpretable to its users and gives them meaningful control over its outputs.

- **M4.1 - Model interpretability (Score: 10):** The Temporal Attention layer produces per-frame weights that can be inspected to understand which frames drove a prediction. This provides a degree of post-hoc interpretability that fully opaque architectures lack.
- **M4.2 - Top-K predictions (Score: 10):** The system exposes Top-5 alongside Top-1. With Top-5 accuracy of 86.46% on WLASL30 (and 90.54% on MS-ASL50), the correct answer is almost always in the shortlist, allowing the user to make the final call rather than accepting a single forced prediction.
- **M4.3 - Reproducibility (Score: 5):** All experiments are driven by versioned YAML configs and produce manifests with git commit, hyperparameters and metrics. The main caveat is that some WLASL videos rely on YouTube URLs that may decay over time; this is also an ethical concern about data provenance, since the original signers did not explicitly consent to use of their videos for machine learning research.

7.3.5 Principle 5: Incorporate Privacy Design Principles

Privacy applies to three distinct stages of the project: training the hand detector on real photographs, training the word recognition model on landmark abstractions and serving inference from a mobile client.

- **M5.1 - Data minimisation through landmarks (training) (Score: 10):** Each video frame is reduced to 63 numerical values that discard the original visual content. Once landmarks are extracted, the original video is no longer needed and the processed data cannot reconstruct the signer's face, body or background.
- **M5.2 - Privacy in the hand detection phase (Score: 5):** Detector training relied on real photographs of identifiable persons (EgoHands, Kaggle Hand). Although these datasets are publicly available for research, no independent privacy audit was conducted on their provenance and the images were used as-is during training.
- **M5.3 - Privacy in the inference pipeline (Score: 0):** The current FastAPI server receives full RGB frames (416×416) from the mobile app, processes them and persists raw frames and cropped regions to disk without encryption or automatic expiration. This is the weakest privacy point. A natural mitigation is to move landmark extraction to the client device so that only the abstract 63-dimensional vector ever leaves the phone.
- **M5.4 - K-anonymity considerations (Score: 5):** WLASL stores no identifying metadata about signers. The landmark representation acts as a generalisation: different persons performing the same sign yield similar feature vectors [36]. A formal k-anonymity algorithm was not applied, but the pipeline achieves a comparable effect for the recognition stage. The protection does not extend to the detector training data nor to the server inference path.

7.3.6 Principle 6: Uphold High Standards of Scientific Excellence

- **M6.1 - Systematic model comparison (word recognition) (Score: 10):** Multiple architectures were trained under matched conditions: CNN+LSTM (pixel-based), LSTM (landmark-

based), plain GRU, GRU with enriched features, GRU + Temporal Attention and a Transformer variant.

- **M6.2 - Systematic model comparison (hand detection) (Score: 10):** The hand detection stage compared YOLOv8n, SSDLite-MobileNetV3 and EfficientDet-D0 on the same combined dataset using standard detection metrics (mAP50, mAP50-95 and latency).
- **M6.3 - Statistical validation (Score: 10):** Stratified 5-Fold Cross-Validation reports mean and standard deviation across folds (63.85% $\pm$ 4.50% Top-1). The modular pipeline contributes additional multi-seed runs on WLASL30 and MS-ASL50 to bound model-init variance. Pearson correlation analysis quantifies the relationship between vocabulary size and accuracy.
- **M6.4 - Literature grounding (Score: 10):** The State of the Art chapter situates each architectural choice (CNN, RNN, LSTM, GRU, Attention, MediaPipe) in the established literature.
- **M6.5 - Honest reporting of limitations (Score: 10):** The project explicitly documents accuracy degradation at scale, the $\sim$69% MediaPipe detection rate on WLASL and the instability observed above 700 classes. Negative results are reported alongside positive findings.

7.3.7 Principle 7: Be Made Available for Uses That Accord with These Principles

- **M7.1 - Intended use clarity (Score: 10):** The project frames the system as a research prototype for ASL word recognition, with assistive technology as the target use case. Scope and limitations are explicit.
- **M7.2 - Misuse potential assessment (Score: 5):** Landmark vectors carry very limited misuse potential (no surveillance or facial recognition is recoverable from 63 floats per frame). The detector and the inference server, however, do process raw images and currently lack access controls and retention policies; this gap should be closed before any deployment beyond the research context.
- **M7.3 - Open research contribution (Score: 5):** The methodology and code are documented in the project report and in the repository. Trained models and processed datasets have not yet been published in a public artefact registry.

7.4 Impact Scale of Ethical Aspects

Table 7.1 consolidates all metrics, their individual scores and the per-principle average.

Principle	ID	Metric	Score	Avg.
P1: Socially Beneficial	M1.1	Addresses a real societal need	10	**8.3**
	M1.2	Effectiveness of the solution	5	
	M1.3	Accessibility of the technology	10	
P2: Avoid Unfair Bias	M2.1	Dataset diversity (word recognition)	5	**6.3**
	M2.2	Dataset diversity (detection & letters)	5	
	M2.3	Input representation fairness	10	
	M2.4	Per-class performance analysis	5	
P3: Safety	M3.1	Cross-validation methodology	10	**8.8**
	M3.2	Scalability testing	10	
	M3.3	Hand detection reliability	10	
	M3.4	Failure mode documentation	5	
P4: Accountability	M4.1	Model interpretability	10	**8.3**
	M4.2	Top-K predictions	10	
	M4.3	Reproducibility	5	
P5: Privacy	M5.1	Data minimisation (landmark training)	10	**5.0**
	M5.2	Privacy in detection phase	5	
	M5.3	Privacy in inference pipeline	0	
	M5.4	K-anonymity considerations	5	
P6: Scientific Excellence	M6.1	Model comparison (word recognition)	10	**10**
	M6.2	Model comparison (hand detection)	10	
	M6.3	Statistical validation	10	
	M6.4	Literature grounding	10	
	M6.5	Honest reporting of limitations	10	
P7: Principled Availability	M7.1	Intended use clarity	10	**6.7**
	M7.2	Misuse potential assessment	5	
	M7.3	Open research contribution	5	
		Overall Average Score		**7.6**

Table 7.1: Ethical evaluation metrics based on Google's AI Principles [35]

7.5 Conclusions of the Ethical Analysis

The project attains a composite ethical maturity score of **7.6/10**, reflecting a high standard of technical integrity with specific architectural risks that remain unresolved.

The evaluation identifies scientific excellence (**10/10**) and safety (**8.8/10**) as the strongest dimensions. These scores reflect the rigour of the multi-seed validation strategy, the systematic reporting of limitations and the reliability of the best-performing configuration within the evaluated scope.

The privacy dimension, rated **5.0/10**, represents the primary area for improvement. This deficit is attributed to the inference pipeline, where live user data is handled with fewer safeguards than in the offline training workflow.

One architectural decision, the use of MediaPipe hand landmarks as input, propagates positive effects across multiple principles: it reduces appearance-based bias (P2), it implements data minimisation that protects identity by construction (P5) and it limits misuse potential because abstract landmarks cannot be repurposed for surveillance (P7). The same advantage, however, does not extend to the upstream stages that operate on raw images. The current FastAPI server receives, processes and persists full RGB frames without encryption or automatic deletion. The hand detector was trained on photographs of identifiable persons. The most actionable mitigation is to relocate landmark extraction to the client device, so that only the 63-dimensional vector ever leaves the phone.

The intermediate scores in bias avoidance (6.3/10) and principled availability (6.7/10) reflect missing formal processes rather than poor design choices: no demographic audit of signers was conducted, the auxiliary detection datasets lack diversity metadata and the server prototype has no access controls or retention policies. Honest reporting of these gaps, including the privacy weakness of the inference pipeline identified in this chapter and the documented accuracy collapse at large vocabularies, is itself an ethical practice aligned with Principle 6 and is a prerequisite for any responsible deployment of the system.

Chapter 8

My Journey at UFV

8.1 The PFG as the Culmination of My University Journey

When I arrived at Universidad Francisco de Vitoria (UFV), I was someone who had just finished secondary school with a clear interest in computer science but a very narrow view of what the field could become. I had always enjoyed working with computers, taking them apart, learning how software worked and trying to build things on my own. There was no family pressure behind the choice; informatics had been a personal vocation from early on. At that point, my main ambition was to become a software developer at one of the large technology companies. I imagined a career almost entirely defined by development.

The choice of UFV was guided by recommendation from people I trusted, and once I attended the first sessions I felt that the academic environment and the human focus of the institution matched what I was looking for. I did not yet fully understand the value of the humanistic approach that the UFV adds to its engineering programmes; that understanding came progressively, over the years.

The first two years of the degree, I was not the most focused version of myself. I tended to leave things until the last moment, trusted that effort under pressure would be enough and approached the work without a real sense of direction. The change came towards the end of the second year and the beginning of the third year. Something shifted in how I understood the relationship between effort, time and results. From that point on, I stopped treating the degree as a sequence of assignments to complete and started treating it as a long-term project to invest in. I became more organised, fought harder for my own objectives and started to take responsibility for the depth of what I was learning, not only for the grade. I am still far from perfect at this, but I can clearly see the distance between who I was at the start and who I am now.

Two people stand out as decisive influences during this journey. The first is Moisés Martínez Muñóz, supervisor of this project, whose way of thinking about Artificial Intelligence and research has been an inspiration throughout the final years of the degree. The second is Olga Peñalba, director of the Escuela Politécnica Superior, who taught the Projects courses in which I was particularly engaged, and through which I obtained a Matrícula de Honor in one of the iterations. Both of them, in different ways, helped me see the engineering profession as something broader and more demanding than the narrow developer career I had pictured at the beginning.

The choice of this specific PFG did not come from a personal connection with the deaf community or with accessibility as a previous interest. It came from a curiosity about computer vision, an attraction to Artificial Intelligence as a discipline and the conviction that the final project of the degree should be ambitious enough to push me into a depth I had not reached yet. The project emerged from my own reflection on what kind of work I wanted to do, not from an externally proposed topic. Choosing sign language recognition gave the project a real and meaningful application, while keeping the technical depth I was looking for. In that sense, the PFG reflects two properties of who I am: a genuine interest in Artificial Intelligence as a field of study, and a willingness to commit fully to an objective once I decide it is the right one.

8.2 Connection with My Professional Future

I am currently doing a software engineering internship at Fintonic in parallel with the development of this project. The experience has been positive and I have learned a lot, but it has also clarified something important: development is enjoyable and I am comfortable doing it, yet it is not where I want my career to settle. The PFG has confirmed that my real interest lies in Artificial Intelligence, in deep learning specifically, and in the engineering practice that makes machine learning systems reliable, reproducible and deployable.

The project has taught me things that neither the degree alone nor the internship alone could have taught me. The first lesson is patience. I had never worked on something this long and this intense, and there were moments of real frustration when progress felt invisible despite hours of work. The second lesson is project management at a scale I had not experienced before. Maintaining a coherent direction across months of experimentation, keeping the code reproducible and the results auditable, and recovering from methodological pivots without losing the thread, are skills I did not have when I started. I believe the final outcome reflects that growth.

Looking ahead, my objective is to pursue a Master's degree in Artificial Intelligence and, ideally, a Doctorate, preferably outside Spain. If a doctoral programme aligned with my interests does not become available, I will look for a role strongly focused on Artificial Intelligence rather than on general software development. The areas I most want to deepen are deep learning and MLOps, because they combine the modelling work I find intellectually rewarding with the engineering rigour needed to bring those models into reliable use. One discovery from this project that I did not anticipate is that I would not mind being a researcher, and I find the idea of teaching at university level genuinely appealing. Both are paths I want to keep open as I continue.

The humanistic dimension of the UFV has played a role here that I did not fully see while it was happening. The way the university trains engineers to think about their work, beyond the purely technical layer, gives an additional perspective that I now consider valuable. It is part of what allows me to see Artificial Intelligence not only as a set of algorithms to optimise, but as a discipline that affects how people communicate, how they are represented in datasets and how they are served by the systems we build. That perspective is visible in the ethical analysis of this project and will remain visible in whatever I do next.

Throughout all of this, the people who have most influenced who I am professionally and personally are my family and the professors who shaped my view of the field. The PFG closes one stage of that journey and opens a new one, in which I want to keep growing, keep specialising in what I love and, in time, contribute something meaningful to the field of Artificial Intelligence.

Chapter 9

Conclusions

9.1 General Conclusions

This Final Degree Project (FDP) studied whether a compact sequence representation of sign movement can retain enough discriminative information to recognise a controlled vocabulary of isolated ASL signs while reducing dependence on raw video appearance. The hypothesis stated in Chapter 6's preamble and traced back to the main objective in Chapter 3 can now be answered.

The answer is **yes, within the tested vocabulary scale, with reproducible stability across two ASL benchmarks and grounded in an explicit ethical framework**.

The landmark-based GRU+Attention pipeline reaches 63.85% Top-1 and 86.46% Top-5 on WLASL30 under Stratified 5-Fold Cross-Validation, and reproduces with comparable seed-to-seed stability on MS-ASL50 (70.20% Top-1, 90.54% Top-5) without any dataset-specific tuning. The pipeline achieves these results while reducing each video frame from a 224×224 RGB tensor to 63 landmark coordinates, a 2,389$\times$ dimensionality reduction that simultaneously discards background appearance, reduces appearance-based bias and protects signer identity by construction.

This work also identifies the boundaries of the approach. Recognition accuracy degrades steadily as the vocabulary grows, from 58.82% Top-1 at 30 classes to 9.76% at 1,500 classes, reflecting the limited per-class data available in WLASL rather than a structural failure of the architecture. The qualitative validation with 15 participants further indicates that the prototype is perceived as useful ($4.5/5$ on perceived utility) and recommendable (Net Promoter Score $+40$), although result comprehension remains the lowest usability dimension and a clear target for interface improvement.

Taken together, the quantitative, qualitative, and ethical evidence validates the prototype, demonstrating its feasibility and reproducibility at a controlled-vocabulary scale while clearly defining its generalization boundaries

9.2 Conclusions regarding Specific Objectives

The six specific objectives defined in Chapter 3 are addressed in the document as follows.

1. **SO1 - Analyse state-of-the-art techniques in computer vision and temporal modelling:** The State of the Art chapter (Chapter 2) reviews convolutional architectures, pose estimation, MediaPipe Hands, recurrent layers, attention mechanisms and Transformer architectures, situating each within the SLR literature. The objective is therefore considered achieved.
2. **SO2 - Define a landmark-based data representation for ASL video clips:** The data preparation pipeline described in Section 5.1 converts raw WLASL and MS-ASL clips into normalised sequences of 21 three-dimensional hand landmarks per frame. The representation is consistent across both datasets and reduces sensitivity to background, lighting and signer appearance. The objective is achieved.
3. **SO3 - Design and compare candidate convolutional and temporal architectures for word-level ASL recognition:** Five locally tuned architecture families were compared under matched conditions on WLASL30 (Section 6.3). The GRU+Attention configuration emerged as the best-performing model under the controlled comparison and was selected as the default for downstream experiments. The objective is achieved.
4. **SO4 - Evaluate the selected recognition pipeline through reproducible experimental protocols:** The selected configuration was evaluated through Stratified 5-Fold Cross-Validation, multi-seed runs at three random seeds and a sequence-length sweep (Sections 6.4 and 6.5). The mean Top-1 of 63.85% $\pm$ 4.50% under Cross-Validation and the seed-to-seed standard deviation of 2.94 pp on the fixed split provide the reproducibility evidence required for the objective. The objective is achieved.
5. **SO5 - Analyse the generalisation and scalability of the recognition pipeline:** The pipeline was retrained on MS-ASL50 without architectural changes (Section 6.6) and on WLASL subsets from 100 to 1,500 classes (Section 6.7). The cross-dataset results confirm that training stability is a property of the pipeline rather than of a specific benchmark, while the scaling experiments quantify the per-class data threshold below which performance degrades. The objective is achieved with the explicit limitation that scaling experiments beyond 30 classes are single-seed.
6. **SO6 - Evaluate the runtime proof of concept and the ethical implications of the recognition approach:** The runtime prototype is described in Chapter 5 and qualitatively validated in Section 6.9 with $N = 15$ participants. The ethical evaluation against Google's AI Principles (Chapter 7) yields an overall maturity score of 7.6/10, identifying the inference pipeline as the primary area for improvement. The objective is achieved.

9.3 Next Steps

The conclusions above identify three frontiers that extend this work, ordered by the type of contribution they would make.

1. **Technical - beyond word-level recognition and on-device miniaturisation:** The natural technical extension is to move from isolated gloss classification to continuous ASL sentence recognition, which remains an open frontier of sign language research. A complementary technical track is to miniaturise the selected GRU+Attention configuration so that the model can run entirely on a mobile device with reduced parameter count, enabling fully on-device inference without server involvement.

2. **Scientific - cross-lingual transfer and broader architectural comparison:** The landmark representation is, by construction, language-agnostic at the geometric level: hand anatomy is shared across sign languages, even when vocabularies are not. A scientific extension is to transfer the pipeline from ASL to Lengua de Signos Española (LSE), British Sign Language (BSL) and other sign languages, evaluating whether the same architecture generalises across linguistic boundaries. A second scientific direction, identified as a limitation in Section 6.8, is to extend the architectural comparison beyond GRU+Attention. In particular, running the Transformer baseline of Section 6.3 on MS-ASL50 and on larger WLASL subsets, using hardware capable of accommodating larger models such as more powerful GPUs, would test whether the recurrent advantage observed at the WLASL30 scale persists in data regimes where Transformer architectures are typically competitive.

3. **Applied - privacy-first edge deployment:** The ethical analysis in Section 7.3 identified the inference pipeline as the weakest privacy point of the current system. The applied next step is therefore to move the entire pipeline on-device, including landmark extraction and recognition, so that only the abstract 63-dimensional vector ever leaves the phone if any data leaves it at all. This closes the privacy gap identified in the ethical evaluation while preserving the recognition capability validated in Section 6.2.

These three directions are not exclusive. A complete continuation of the work would combine continuous-sentence modelling with cross-lingual evaluation under a privacy-first deployment configuration, building on the foundations established in this Final Degree Project.

Bibliography

[1] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1459–1469.

[2] H. R. V. Joze and O. Koller, "Ms-asl: A large-scale data set and benchmark for understanding american sign language," in *British Machine Vision Conference (BMVC)*, 2019.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[6] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 818–833.

[7] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," in *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.

[8] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990. DOI: 10.1109/5.58337.

[9] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994. DOI: 10.1109/72.279181.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *NIPS 2014 Deep Learning Workshop*, 2014.

[12] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[14] O. Koller, "Quantitative survey of the state of the art in sign language recognition," *arXiv preprint arXiv:2008.09918*, 2020.

[15] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," vol. 164, Elsevier, 2021, p. 113 794.

[16] D. Bragg, O. Koller, M. Bellard, L. Berke, P. Boudreault, A. Braffort, N. Caselli, M. Huenerfauth, H. Kacorri, T. Verhoef, et al., "Sign language recognition, generation, and translation: An interdisciplinary perspective," pp. 16–31, 2019.

[17] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[18] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *European Conference on Computer Vision (ECCV)*, Springer, 2016, pp. 21–37.

[19] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 781–10 790.

[20] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," pp. 779–788, 2016. DOI: 10.1109/CVPR.2016.91.

[21] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[22] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, https://github.com/ultralytics/ultralytics, Accessed: 2025, 2023.

[23] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 023–10 033.

[24] K. Yin and J. Read, "Better sign language translation with STMC-transformer," in *Proceedings of the 28th International Conference on Computational Linguistics*, International Com-

mittee on Computational Linguistics, 2020, pp. 5975–5989. DOI: 10.18653/v1/2020.coling-main.525.

[25] M. Bohacek and M. Hruz, "Sign pose-based transformer for word-level sign language recognition," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, 2022, pp. 182–191. DOI: 10.1109/WACVW54805.2022.00024.

[26] J. Eunice, Y. Sei, and D. J. Hemanth, "Sign2pose: A pose-based approach for gloss prediction using a transformer model," *Sensors*, vol. 23, no. 5, p. 2853, 2023. DOI: 10.3390/s23052853.

[27] S. Albanie, G. Varol, L. Momeni, T. Afouras, J. S. Chung, N. Fox, and A. Zisserman, "Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues," in *European Conference on Computer Vision (ECCV)*, Springer, 2020, pp. 35–53. DOI: 10.1007/978-3-030-58621-8_3.

[28] E. Ries, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. New York: Crown Business, 2011.

[29] S. Bambach, S. Lee, D. J. Crandall, and C. Yu, "Lending a hand: Detecting hands and recognizing activities in complex egocentric interactions," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1949–1957.

[30] A. Nikkhah, *Hand dataset*, https://www.kaggle.com/datasets/armannikkhah/hand-dataset, Accessed: 2025, 2023.

[31] TechnicianLP, *Sign language mnist dataset*, https://www.kaggle.com/datamunge/sign-language-mnist, Accessed: 2025, 2017.

[32] Akash, *Asl alphabet dataset*, https://www.kaggle.com/grassknoted/asl-alphabet, Accessed: 2025, 2018.

[33] SignToText Project, *Signtotext qualitative validation survey*, Google Forms survey: https://forms.gle/zCqmetDjqMUG46Eh9, Questionnaire used for the qualitative validation of the SignToText demo, 2026.

[34] S. Pichai, *Ai at google: Our principles*, https://blog.google/technology/ai/ai-principles/, Published June 7, 2018. Accessed: 2026, 2018.

[35] Google, "Ai principles progress update 2023," Google, Tech. Rep., 2023, 5th edition of Google's annual AI Principles progress report. Accessed: 2026.

[36] L. Sweeney, "K-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002. DOI: 10.1142/S0218488502001648.

Appendix A

Additional Experimental Figures

This appendix contains large experimental figures that are required for evaluation but are not legible at the standard inline size used in the main chapters. They are referenced from Chapter 6 where their corresponding results are discussed.

A.1 Large Confusion Matrix

Figure A.1 shows the row-normalised confusion matrix for the MS-ASL50 seed-2026 run. The matrix is placed here because the 50-class layout requires a full page to make the class labels and off-diagonal errors readable.

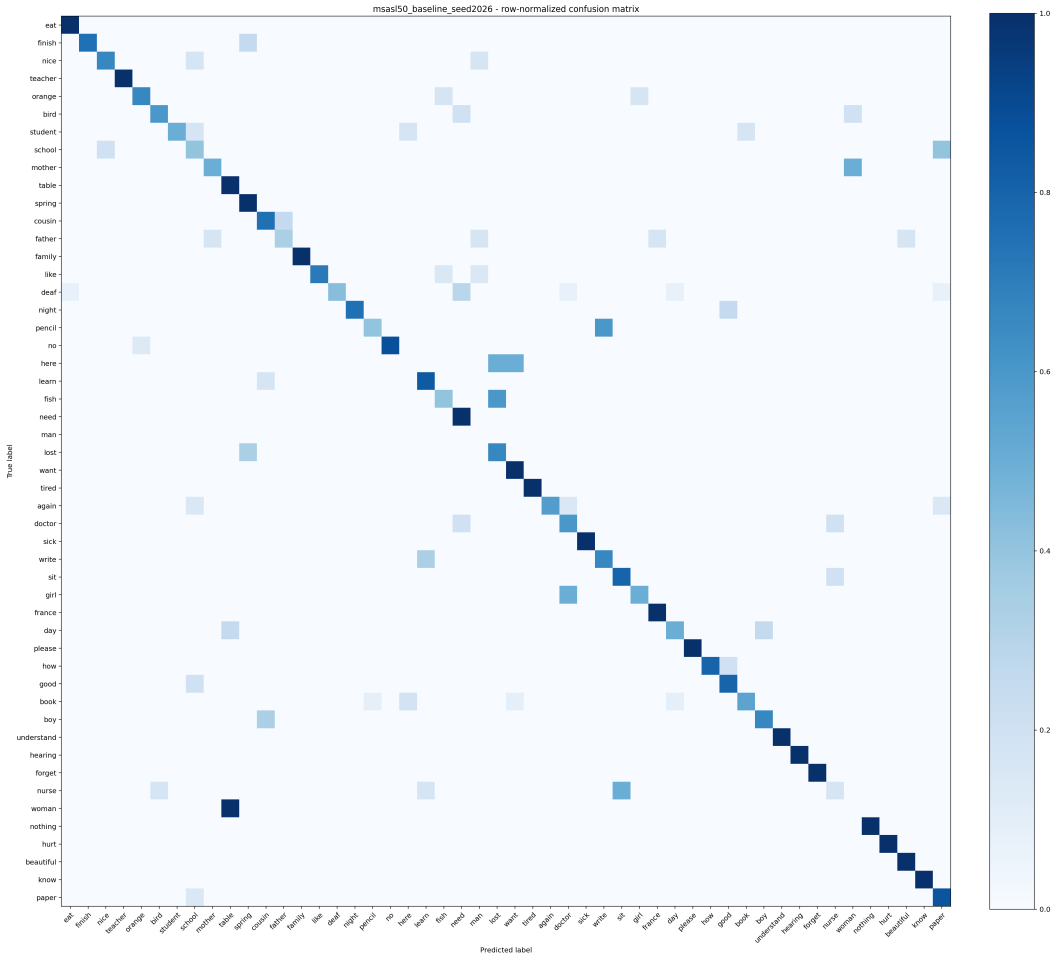


Figure A.1: Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). Source: self-created from experiment outputs.

A.2 Vocabulary Scaling Diagnostic

Figure A.2 shows the parallel-coordinates diagnostic used in Section 6.7. Each line corresponds to one vocabulary-scaling run, projected onto mean videos per class, final train accuracy, validation Top-1, test Top-1 and test Top-5.

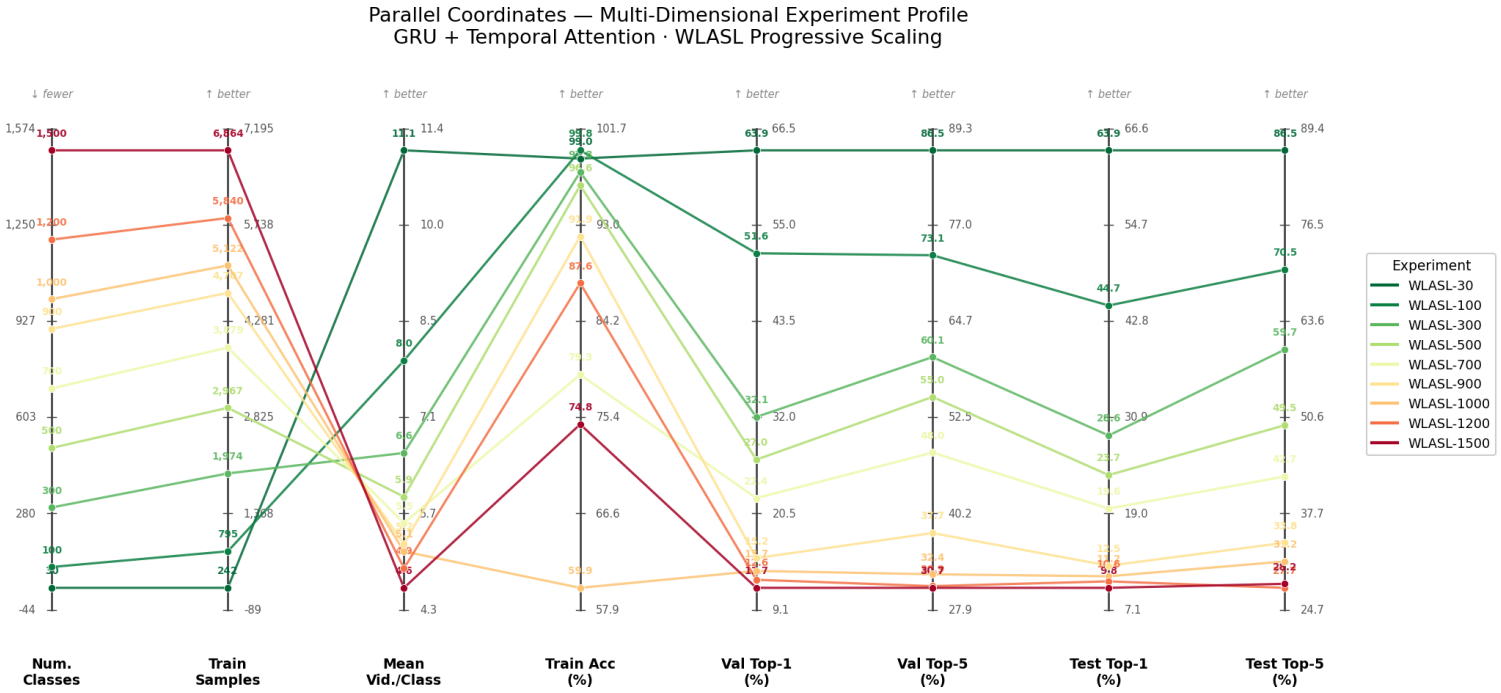


Figure A.2: Parallel-coordinates view of the selected GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green for smaller vocabularies to red for larger vocabularies. Source: self-created from experiment outputs.

Appendix B

User Manual

This appendix describes the practical procedure required to install, prepare and execute the prototype developed in this FDP. All commands are intended to be executed from the repository root directory, unless another directory is explicitly indicated.

B.1 Repository Layout

The project repository is organised around the main development artefacts:

- `src/slr/`: reusable Python package for landmark datasets, models, training, evaluation and experiment tracking.
- `scripts/`: command-line entry points for training, evaluation, MS-ASL subset construction and landmark extraction.
- `configs/`: YAML experiment configurations. Base files are stored under `configs/base/` and concrete runs under `configs/experiments/`.
- `05_experiments/`: experiment outputs, including metrics, figures, reports, manifests and local checkpoints when available.
- `06_datasets/`: local datasets, landmark files and dataset-specific metadata. This directory is intentionally not fully tracked in Git because of size and licensing constraints.
- `04_server/`: FastAPI inference server used by the runtime prototype.
- `03_app/signtotext-expo/`: Expo/React Native mobile proof of concept.

B.2 Environment Setup

Python 3.10 or newer is required. The training code can run on CPU, but a CUDA-capable NVIDIA GPU is recommended for reproducible training times. The environment used in the project is described in Section 5.1.5.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[landmarks,dev]"
```

The editable install exposes the `slr` package and installs the core training dependencies. The `landmarks` extra installs MediaPipe and OpenCV, which are required for landmark extraction and inference. The `dev` extra installs the test runner.

The FastAPI prototype has a small additional requirements file:

```
pip install -r 04_server/requirements.txt
```

The mobile prototype uses Expo and pnpm:

```
cd 03_app/signtotext-expo
pnpm install
```

B.3 Dataset Preparation

Large datasets and downloaded videos are not included in the repository. A local workspace must therefore restore or regenerate the required data under `06_datasets/`. The expected landmark paths used by the main configurations include:

```
06_datasets/wlasl/landmarks/wlasl100_filtered30/
06_datasets/MS-ASL/subsets/msasl50/
```

For WLASL experiments, the training configuration expects a directory split with `train/`, `val/` and `test/` folders plus a `metadata.json` file with the gloss mapping. The baseline configuration points to:

```
06_datasets/wlasl/landmarks/wlasl100_filtered30/metadata.json
```

For MS-ASL, the original annotation files are stored under `06_datasets/MS-ASL/`. Because the dataset references online videos, the local available subset depends on which source videos can still be downloaded. The selected MS-ASL50 subset can be reconstructed from the local audit with:

```
python scripts/build_msasl_subset.py \
  --audit-csv 06_datasets/MS-ASL/audit/audit_per_class.csv \
  --audit-videos-csv 06_datasets/MS-ASL/audit/audit_videos.csv \
  --output-dir 06_datasets/MS-ASL/subsets/msasl50 \
  --n-classes 50 \
  --min-train 20 \
  --min-val 3 \
  --min-test 3
```

Once the subset metadata and source videos are available, hand landmarks are extracted with:

```
python scripts/extract_landmarks_msasl.py \
  --subset-metadata \
  06_datasets/MS-ASL/subsets/msasl50/metadata.json \
  --videos-dir 06_datasets/MS-ASL/videos \
  --output-dir \
```

```
  06_datasets/MS-ASL/subsets/msasl50/landmarks \
  --max-frames 64 \
  --min-detection-rate 0.3 \
  --skip-existing
```

This process writes per-clip `.npy` files containing 21 three-dimensional hand landmarks per valid frame, together with extraction reports and metadata used by the training pipeline.

B.4 Training

Training is driven by YAML configuration files. A standard WLASL30 run is launched with:

```
python scripts/train.py \
  --config configs/experiments/wlasl30_baseline.yaml
```

The same entry point is used for the MS-ASL50 multi-seed runs:

```
python scripts/train.py \
  --config configs/experiments/msasl50_baseline/seed42.yaml
python scripts/train.py \
  --config configs/experiments/msasl50_baseline/seed123.yaml
python scripts/train.py \
  --config configs/experiments/msasl50_baseline/seed2026.yaml
```

The device can be forced when required:

```
python scripts/train.py \
  --config configs/experiments/wlasl30_baseline.yaml \
  --device cpu
python scripts/train.py \
  --config configs/experiments/wlasl30_baseline.yaml \
  --device cuda
```

Each completed run writes an experiment directory under `05_experiments/`. The most relevant artifacts are `best_model.pt`, `training_history.json`, `results.json`, `per_class_metrics.csv`, confusion matrices and `experiment_manifest.json`. The manifest is the main reproducibility artifact because it records the resolved configuration, git state, hardware information, runtime and final metrics.

B.5 Evaluation

An existing checkpoint can be evaluated on a split directory with:

```
python scripts/evaluate.py \
  --checkpoint \
  05_experiments/final_assault/best_model_attention.pt \
  --metadata \
  06_datasets/wlasl/landmarks/wlasl30/metadata.json \
  --dataset-split 06_datasets/wlasl/landmarks/wlasl30/test \
  --output-dir 05_experiments/final_assault/eval_test \
```

```
--architecture gru_attention
```

The evaluation command reports Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy, and writes the corresponding plots and tables to the selected output directory.

B.6 Inference Server

The FastAPI server is started from the server directory:

```
cd 04_server
python main.py
```

By default, the server listens on port 8000. It expects the checkpoint and metadata paths declared in `04_server/main.py` to exist locally. In particular, the production registry entry uses the GRU+Attention WLASL30 checkpoint under `05_experiments/final_assault/` and the corresponding WLASL30 metadata under `06_datasets/`.

The available HTTP endpoints are:

- `GET /health`: server status and loaded model information.
- `GET /models`: available model registry entries.
- `GET /classes`: class list for the selected model.
- `POST /predict`: isolated-word prediction from an uploaded video.
- `POST /predict-sequence`: heuristic continuous-sequence prediction.

The isolated prediction endpoint accepts a video upload and returns Top-K gloss predictions, confidence scores, the detection rate and runtime metadata.

B.7 Mobile Prototype

After installing the mobile dependencies, the Expo application can be started with:

```
cd 03_app/signtotext-expo
pnpm start
```

The application records or selects a short video, sends it to the FastAPI server, displays the Top-K predictions and can read the Top-1 result through text-to-speech. For a physical device, the server URL configured in the mobile client must point to the computer running FastAPI, not to `localhost` on the phone.

B.8 Smoke Tests

The Python unit tests do not require the full video datasets:

```
pytest
```

A minimal runtime check consists of starting the server and requesting the health endpoint:

```
curl http://localhost:8000/health
```

If the server starts but model loading fails during prediction, the most likely cause is a missing local checkpoint, missing `metadata.json` file or missing MediaPipe `hand_landmarker.task` file in `04_server/`.

Appendix C

Declaration of the Use of AI Systems

In the preparation of this Final Degree Project, artificial intelligence systems have been used responsibly at different stages of the process. This appendix documents how, where and for what purpose these systems contributed to the work, in accordance with UFV regulations for the preparation and defence of Final Degree Projects.

Systems used

The following systems were used at different points during the project:

- **Claude** (Anthropic), versions 3.7 Sonnet, 4.5 Sonnet and 4.7 Opus, accessed through the Claude web interface and through Claude Code as a command-line coding agent. Both Sonnet and Opus variants of Claude Code were used depending on the complexity of the task.
- **ChatGPT** (OpenAI), used through its web interface for occasional rewriting tasks and for resolving specific code questions and runtime errors.
- **OpenAI Codex CLI**, used directly from the terminal as an additional coding agent.
- **Cursor** (IDE with integrated AI), where both Claude and GPT models were used as the underlying language models for in-editor assistance.
- **Gemini** (Google), used exclusively for the generation of specific illustrative images. Every figure produced with Gemini is explicitly attributed in its caption within the main body of the document.

Stages of use

Literature analysis. AI systems played a minor role in this stage. The bibliographic search was conducted mainly through Google Scholar and following the recommendations of the project supervisor. AI was used occasionally to summarise individual papers when a quick conceptual overview was useful, but sources were always read and verified directly before being cited in this work.

Software development and experimentation. The implementation of the recognition pipeline, training scripts and evaluation tooling was carried out following a mixed workflow. The author wrote the initial structure of each module, and AI coding agents (mainly Claude Code, complemented by OpenAI Codex CLI and in-editor assistance through Cursor) were asked to refine specific blocks, review existing code or generate new components within a defined scope. Every AI-generated or AI-modified block was subsequently reviewed, tested and adjusted by the author before being committed to the repository. The design decisions, the methodological pivot from pixel-based to landmark-based recognition, the choice of architectures, the experimental protocols and the interpretation of results were taken by the author. AI agents never made autonomous commits.

Data curation. AI assistance was used to support data preparation scripts, including dataset reconstruction routines, MS-ASL download auditing and landmark extraction utilities. The dataset selection criteria, the threshold choices for subset construction (MS-ASL50, WLASL30) and the final validation of the processed data were decisions taken by the author.

Writing of the document. AI systems were used during the writing of this report in two distinct ways. First, for the initial drafting of certain passages, where the author provided notes, bullet points and decisions, and AI systems produced prose drafts that were then revised, restructured and rewritten by the author to match the voice and content of the work. Second, for grammatical and stylistic revision, including consistency checks, identification of awkward phrasings and reorganisation of paragraphs within chapters based on supervisor feedback. The technical content, the experimental claims, the structure of arguments and the final wording of every chapter were decided by the author.

Visualisation. Most figures in this work were produced by the author using TikZ, GanttProject or by adapting figures from cited academic sources. A small number of illustrative images were generated using Gemini and are identified explicitly in their captions.

Academic responsibility

Throughout every stage of the project, academic judgement was maintained. The use of AI systems did not replace the conceptual work of the project: the research questions, the hypothesis, the methodological design, the interpretation of experimental results and the ethical evaluation were the author's own. AI systems were used as productivity tools and as drafting and revision assistants, in the same way that a researcher might use a spell checker, an IDE autocomplete, a reference manager or a peer reviewer. The originality and quality of the work, and the responsibility for every claim contained in it, rest with the author.

AI contribution summary (CRediT statement)

- **Conceptualization:** Author.
- **Methodology:** Author.
- **Software:** Author, with AI assistance (Claude Code, OpenAI Codex CLI, Cursor with Claude and GPT models).
- **Validation:** Author.

- **Formal analysis:** Author.
- **Investigation:** Author.
- **Resources:** Author.
- **Data curation:** Author, with AI assistance.
- **Writing – original draft preparation:** Author, with AI assistance.
- **Writing – review and editing:** Author, with AI assistance.
- **Visualization:** Author. Specific illustrative figures generated with Gemini are individually attributed in their captions.
- **Supervision:** Moisés Martínez Muñoz, project supervisor.
- **Project administration:** Author.

All uses of AI in this Final Degree Project that produced specific content referenced in the main text are cited at the appropriate location, in accordance with the UFV citation guidelines.